

C++ STL 完整自学手册

基于《C++ Primer Plus》第16章

目录

- 前言
- 第一部分：string类详解
- 第二部分：智能指针
- 第三部分：STL基础
- 第四部分：容器详解
- 第五部分：迭代器深入
- 第六部分：算法详解
- 第七部分：函数对象
- 第八部分：实战练习
- 附录

前言

什么是STL?

STL (Standard Template Library, 标准模板库) 是C++标准库的核心组成部分。它提供了一套完整的、经过精心设计的模板类和函数，让您能够高效地处理数据和算法。

为什么要学习STL?

- 提高编程效率：避免重复造轮子
- 代码质量保证：经过广泛测试的高质量实现
- 标准化：跨平台、跨编译器的一致性
- 性能优化：经过优化的算法实现

学习路线图

string类 → 智能指针 → 容器 → 迭代器 → 算法 → 函数对象 → 综合应用

第一部分：string类详解

1.1 为什么需要string类?

在学习STL之前，让我们先了解string类。传统的C风格字符串使用char数组，存在许多问题：

cpp

```
// C风格字符串的问题
char name1[20] = "Alice";
char name2[20] = "Bob";
// strcat(name1, name2); // 可能导致缓冲区溢出!
// if (name1 == name2) // 这比较的是地址, 不是内容!
```

string类解决了这些问题:

cpp

```
#include <string>
string name1 = "Alice";
string name2 = "Bob";
string fullName = name1 + " " + name2; // 安全的字符串连接
if (name1 == name2) // 正确的内容比较
    cout << "Names are equal" << endl;
```

1.2 string类的构造方法

string类提供了多种构造字符串的方法:

cpp

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    // 1. 默认构造函数
    string str1; // 空字符串

    // 2. 从C风格字符串构造
    string str2("Hello World");

    // 3. 复制构造
    string str3(str2);

    // 4. 重复字符构造
    string str4(5, 'A'); // "AAAAA"

    // 5. 子字符串构造
    string str5(str2, 6, 5); // 从位置6开始，取5个字符: "World"

    // 6. 迭代器范围构造
    string str6(str2.begin(), str2.begin() + 5); // "Hello"

    // 输出所有字符串
    cout << "str1: " << str1 << " " << endl;
    cout << "str2: " << str2 << endl;
    cout << "str3: " << str3 << endl;
    cout << "str4: " << str4 << endl;
    cout << "str5: " << str5 << endl;
    cout << "str6: " << str6 << endl;

    return 0;
}
```

1.3 string类的基本操作

字符串赋值和连接

cpp

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "Hello";
    string str2 = "World";

    // 字符串连接
    string result1 = str1 + " " + str2; // "Hello World"

    // 使用 += 运算符
    str1 += " ";
    str1 += str2; // str1 现在是 "Hello World"

    // 可以连接不同类型
    str1 += '!'; // 添加字符
    str1 += " 2024"; // 添加C风格字符串

    cout << "result1: " << result1 << endl;
    cout << "str1: " << str1 << endl;

    return 0;
}
```

字符串比较

cpp

```

#include <iostream>
#include <string>
using namespace std;

int main() {
    string str1 = "apple";
    string str2 = "banana";
    string str3 = "apple";

    // 比较操作符
    cout << "str1 == str3: " << (str1 == str3) << endl; // 1 (true)
    cout << "str1 < str2: " << (str1 < str2) << endl; // 1 (true, 字典序)
    cout << "str1 > str2: " << (str1 > str2) << endl; // 0 (false)

    // 也可以与C风格字符串比较
    cout << "str1 == \"apple\": " << (str1 == "apple") << endl; // 1

    return 0;
}

```

1.4 字符串输入输出

不同的输入方式

```

cpp

#include <iostream>
#include <string>
using namespace std;

int main() {
    string word, line;

    cout << "输入一个单词: ";
    cin >> word; // 读取到空格为止
    cout << "你输入的单词是: " << word << endl;

    cin.ignore(); // 清除输入缓冲区中的换行符

    cout << "输入一行文本: ";
    getline(cin, line); // 读取整行, 包括空格
    cout << "你输入的行是: " << line << endl;

    return 0;
}

```

自定义分隔符

```
cpp

#include <iostream>
#include <string>
using namespace std;

int main() {
    string data;

    cout << "输入用冒号分隔的数据: ";
    getline(cin, data, ':'); // 以冒号为分隔符
    cout << "读取到: " << data << endl;

    return 0;
}
```

1.5 字符串搜索和操作

查找操作

```
cpp
```

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "Hello World! Welcome to the World of C++!";

    // 查找子字符串
    size_t pos = text.find("World");
    if (pos != string::npos) {
        cout << "找到 'World' 在位置: " << pos << endl;
    }

    // 从指定位置开始查找
    pos = text.find("World", pos + 1);
    if (pos != string::npos) {
        cout << "第二个 'World' 在位置: " << pos << endl;
    }

    // 查找字符
    pos = text.find('!');
    cout << "第一个 '!' 在位置: " << pos << endl;

    // 反向查找
    pos = text.rfind("o");
    cout << "最后一个 'o' 在位置: " << pos << endl;

    return 0;
}
```

字符串修改

cpp

```
#include <iostream>
#include <string>
using namespace std;

int main() {
    string text = "Hello World";

    // 替换字符
    text[6] = 'w'; // "Hello world"
    cout << "替换后: " << text << endl;

    // 插入字符串
    text.insert(5, " Beautiful"); // "Hello Beautiful world"
    cout << "插入后: " << text << endl;

    // 删除部分字符串
    text.erase(5, 10); // 从位置5开始删除10个字符
    cout << "删除后: " << text << endl;

    // 替换子字符串
    size_t pos = text.find("world");
    if (pos != string::npos) {
        text.replace(pos, 5, "C++"); // 将"world"替换为"C++"
    }
    cout << "替换后: " << text << endl;

    return 0;
}
```

1.6 实用的string方法

cpp


```
#include <iostream>
#include <string>
#include <algorithm>
#include <cctype>
using namespace std;

int main() {
    string text = " Hello World ";

    // 获取长度
    cout << "长度: " << text.length() << endl;
    cout << "大小: " << text.size() << endl; // 与length()相同

    // 检查是否为空
    cout << "是否为空: " << text.empty() << endl;

    // 获取子字符串
    string sub = text.substr(2, 5); // 从位置2开始，取5个字符
    cout << "子字符串: " << sub << " " << endl;

    // 转换为C风格字符串
    const char* cstr = text.c_str();
    cout << "C风格字符串: " << cstr << endl;

    // 清空字符串
    string temp = text;
    temp.clear();
    cout << "清空后长度: " << temp.length() << endl;

    return 0;
}
```

1.7 string类实战：单词游戏

让我们创建一个简单的猜单词游戏来实践string类的使用：

cpp

```
#include <iostream>
#include <string>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <algorithm>
using namespace std;

class WordGame {
private:
    vector<string> words;
    string currentWord;
    string guessedWord;
    int wrongGuesses;
    const int maxWrongGuesses = 6;

public:
    WordGame() {
        // 初始化单词列表
        words = {"programming", "computer", "algorithm", "function", "variable"};
        srand(time(nullptr));
        wrongGuesses = 0;
    }

    void startNewGame() {
        // 随机选择一个单词
        int index = rand() % words.size();
        currentWord = words[index];

        // 初始化猜测状态
        guessedWord = string(currentWord.length(), '-');
        wrongGuesses = 0;

        cout << "欢迎来到猜单词游戏！ " << endl;
        cout << "单词有 " << currentWord.length() << " 个字母" << endl;
        displayStatus();
    }

    void displayStatus() {
        cout << "\n当前状态: " << guessedWord << endl;
        cout << "错误次数: " << wrongGuesses << "/" << maxWrongGuesses << endl;
        cout << "请输入一个字母: ";
    }

    bool makeGuess(char letter) {
        letter = tolower(letter); // 转换为小写
```

```

    bool found = false;

    // 检查字母是否在单词中
    for (size_t i = 0; i < currentWord.length(); i++) {
        if (currentWord[i] == letter && guessedWord[i] == '-') {
            guessedWord[i] = letter;
            found = true;
        }
    }

    if (!found) {
        wrongGuesses++;
        cout << "抱歉，字母 '" << letter << "' 不在单词中。" << endl;
    } else {
        cout << "太好了！字母 '" << letter << "' 在单词中。" << endl;
    }

    return found;
}

bool isGameWon() {
    return guessedWord == currentWord;
}

bool isGameLost() {
    return wrongGuesses >= maxWrongGuesses;
}

void revealWord() {
    cout << "正确答案是: " << currentWord << endl;
}
};

int main() {
    WordGame game;
    char letter;
    char playAgain;

    do {
        game.startNewGame();

        while (!game.isGameWon() && !game.isGameLost()) {
            cin >> letter;
            game.makeGuess(letter);

            if (!game.isGameWon() && !game.isGameLost()) {
                game.displayStatus();
            }
        }
    } while (playAgain != 'n');
}

```

```

    }
}

if (game.isGameWon()) {
    cout << "\n恭喜你！你猜对了！ " << endl;
} else {
    cout << "\n游戏结束！ " << endl;
    game.revealWord();
}

cout << "想再玩一次吗？(y/n): ";
cin >> playAgain;

} while (playAgain == 'y' || playAgain == 'Y');

cout << "谢谢游戏！ " << endl;
return 0;
}

```

第二部分：智能指针

2.1 为什么需要智能指针？

在传统C++中，使用new和delete管理内存容易出现问题：

```

cpp

// 问题代码示例
void problematicFunction() {
    int* ptr = new int(42);

    // 如果这里抛出异常，内存就泄漏了！
    if (someCondition) {
        throw runtime_error("Something went wrong");
    }

    delete ptr; // 可能永远执行不到
}

```

智能指针解决了这个问题：

```

cpp

```

```
#include <memory>
```

```
void safeFunction() {  
    std::unique_ptr<int> ptr(new int(42));  
    // 或者更好的方式:  
    auto ptr2 = std::make_unique<int>(42);  
  
    // 即使抛出异常, 智能指针的析构函数也会自动释放内存  
    if (someCondition) {  
        throw runtime_error("Something went wrong");  
    }  
  
    // 函数结束时, ptr 自动释放内存  
}
```

2.2 unique_ptr详解

`unique_ptr`是独占式智能指针，确保只有一个指针拥有资源。

基本使用

```
cpp
```

```
#include <iostream>
#include <memory>
using namespace std;

int main() {
    // 创建unique_ptr的几种方式

    // 方式1：使用构造函数
    unique_ptr<int> ptr1(new int(42));

    // 方式2：使用make_unique (推荐)
    auto ptr2 = make_unique<int>(100);

    // 方式3：创建数组
    auto arr = make_unique<int[]>(10);

    // 使用智能指针
    cout << "ptr1指向的值: " << *ptr1 << endl;
    cout << "ptr2指向的值: " << *ptr2 << endl;

    // 填充数组
    for (int i = 0; i < 10; i++) {
        arr[i] = i * i;
    }

    // 输出数组
    for (int i = 0; i < 10; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
    // 程序结束时，所有智能指针自动释放内存
}
```

unique_ptr的移动语义

cpp

```

#include <iostream>
#include <memory>
#include <vector>
using namespace std;

unique_ptr<int> createNumber(int value) {
    return make_unique<int>(value); // 可以返回unique_ptr
}

int main() {
    auto ptr1 = make_unique<int>(42);

    // unique_ptr不能复制, 但可以移动
    // auto ptr2 = ptr1; // 错误! 不能复制
    auto ptr2 = move(ptr1); // 正确! 移动所有权

    // 现在ptr1为空, ptr2拥有资源
    cout << "ptr1是否为空: " << (ptr1 == nullptr) << endl;
    cout << "ptr2指向的值: " << *ptr2 << endl;

    // 从函数返回unique_ptr
    auto ptr3 = createNumber(100);
    cout << "ptr3指向的值: " << *ptr3 << endl;

    // 可以存储在容器中
    vector<unique_ptr<int>> vec;
    vec.push_back(make_unique<int>(1));
    vec.push_back(make_unique<int>(2));
    vec.push_back(make_unique<int>(3));

    cout << "容器中的值: ";
    for (const auto& ptr : vec) {
        cout << *ptr << " ";
    }
    cout << endl;

    return 0;
}

```

2.3 shared_ptr详解

`shared_ptr`是共享式智能指针，允许多个指针共享同一个资源。

基本使用

```

#include <iostream>
#include <memory>
using namespace std;

class Person {
public:
    string name;
    int age;

    Person(const string& n, int a) : name(n), age(a) {
        cout << "Person " << name << " 被创建" << endl;
    }

    ~Person() {
        cout << "Person " << name << " 被销毁" << endl;
    }

    void introduce() {
        cout << "我是 " << name << ", 今年 " << age << " 岁" << endl;
    }
};

int main() {
    // 创建shared_ptr
    auto person1 = make_shared<Person>("Alice", 25);
    cout << "引用计数: " << person1.use_count() << endl;

    {
        // 在新的作用域中创建另一个shared_ptr
        auto person2 = person1; // 共享同一个Person对象
        cout << "引用计数: " << person1.use_count() << endl;

        person2->introduce();

        // person2在这里被销毁，但Person对象还存在
    }

    cout << "引用计数: " << person1.use_count() << endl;
    person1->introduce();

    return 0;
    // person1被销毁，Person对象也被销毁
}

```

shared_ptr的实际应用场景


```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

class Document {
public:
    string content;

    Document(const string& text) : content(text) {
        cout << "Document创建: " << content.substr(0, 20) << "..." << endl;
    }

    ~Document() {
        cout << "Document销毁: " << content.substr(0, 20) << "..." << endl;
    }
};

class Editor {
private:
    shared_ptr<Document> doc;

public:
    Editor(shared_ptr<Document> document) : doc(document) {}

    void edit(const string& newContent) {
        doc->content = newContent;
        cout << "编辑器修改了文档" << endl;
    }
};

class Viewer {
private:
    shared_ptr<Document> doc;

public:
    Viewer(shared_ptr<Document> document) : doc(document) {}

    void view() {
        cout << "查看器显示: " << doc->content << endl;
    }
};

int main() {
    // 创建一个文档
    auto document = make_shared<Document>("这是一个重要的文档内容");
```

```
// 多个对象可以共享同一个文档
Editor editor(document);
Viewer viewer1(document);
Viewer viewer2(document);

cout << "文档引用计数: " << document.use_count() << endl;

// 查看原始内容
viewer1.view();

// 编辑文档
editor.edit("这是修改后的文档内容");

// 所有查看器都能看到修改后的内容
viewer1.view();
viewer2.view();

return 0;
// 所有对象销毁后，文档也会被自动销毁
}
```

2.4 weak_ptr详解

`weak_ptr`是弱引用智能指针，可以观察`shared_ptr`管理的对象，但不影响引用计数。

cpp

```
#include <iostream>
#include <memory>
using namespace std;

class Child;

class Parent {
public:
    string name;
    shared_ptr<Child> child;

    Parent(const string& n) : name(n) {
        cout << "Parent " << name << " 被创建" << endl;
    }

    ~Parent() {
        cout << "Parent " << name << " 被销毁" << endl;
    }
};

class Child {
public:
    string name;
    weak_ptr<Parent> parent; // 使用weak_ptr避免循环引用

    Child(const string& n) : name(n) {
        cout << "Child " << name << " 被创建" << endl;
    }

    ~Child() {
        cout << "Child " << name << " 被销毁" << endl;
    }

    void sayHello() {
        // 使用weak_ptr时需要先检查对象是否还存在
        if (auto p = parent.lock()) {
            cout << "Child " << name << " 说: 你好, " << p->name << "! " << endl;
        } else {
            cout << "Child " << name << " 说: 父母已经不存在了" << endl;
        }
    }
};

int main() {
    auto parent = make_shared<Parent>("王爸爸");
    auto child = make_shared<Child>("小明");
```

```
// 建立父子关系
parent->child = child;
child->parent = parent; // weak_ptr不增加引用计数

cout << "Parent引用计数: " << parent.use_count() << endl; // 应该是1

child->sayHello();

// 重置parent，模拟父母对象被销毁
parent.reset();

// child仍然存在，但parent已经被销毁
child->sayHello();

return 0;
}
```

2.5 智能指针最佳实践

选择合适的智能指针

cpp

```
#include <iostream>
#include <memory>
#include <vector>
using namespace std;

// 1. 使用unique_ptr表示独占所有权
unique_ptr<int> createUniqueResource() {
    return make_unique<int>(42);
}

// 2. 使用shared_ptr表示共享所有权
shared_ptr<string> createSharedResource() {
    return make_shared<string>("共享资源");
}

// 3. 函数参数的最佳实践
void processRawPointer(int* ptr) {
    // 不获取所有权，只是使用
    if (ptr) {
        cout << "处理值: " << *ptr << endl;
    }
}

void processUniquePtr(unique_ptr<int> ptr) {
    // 接管所有权
    cout << "接管并处理值: " << *ptr << endl;
}

void processSharedPtr(shared_ptr<string> ptr) {
    // 共享所有权
    cout << "共享并处理: " << *ptr << endl;
}

int main() {
    // 创建资源
    auto uniqueRes = createUniqueResource();
    auto sharedRes = createSharedResource();

    // 传递给不获取所有权的函数
    processRawPointer(uniqueRes.get()); // 使用get()获取原始指针

    // 传递给需要所有权的函数
    processUniquePtr(move(uniqueRes)); // 移动所有权
    // uniqueRes现在为空

    // 共享所有权
```

```
processSharedPtr(sharedRes); // 复制shared_ptr  
// sharedRes仍然有效  
  
cout << "sharedRes仍然可用: " << *sharedRes << endl;  
  
return 0;  
}
```

自定义删除器

cpp

```

#include <iostream>
#include <memory>
#include <cstdio>
using namespace std;

// 自定义删除器用于FILE*
void fileDeleter(FILE* fp) {
    if (fp) {
        cout << "关闭文件" << endl;
        fclose(fp);
    }
}

int main() {
    // 使用自定义删除器管理FILE*
    unique_ptr<FILE, decltype(&fileDeleter)> file(
        fopen("test.txt", "w"),
        fileDeleter
    );

    if (file) {
        fprintf(file.get(), "Hello, World!\n");
        cout << "写入文件成功" << endl;
    }

    // 也可以使用lambda作为删除器
    auto customDeleter = [](int* ptr) {
        cout << "自定义删除器释放内存" << endl;
        delete ptr;
    };

    unique_ptr<int, decltype(customDeleter)> ptr(new int(42), customDeleter);

    return 0;
    // 文件和内存都会被正确释放
}

```

第三部分：STL基础

3.1 STL的组成部分

STL由四个主要组件组成：

1. **容器 (Containers)**：存储数据的数据结构
2. **迭代器 (Iterators)**：访问容器元素的通用接口

3. **算法 (Algorithms)**: 操作容器数据的函数

4. **函数对象 (Function Objects)**: 可调用的对象

让我们用一个简单的例子来理解这些组件:

```
cpp

#include <iostream>
#include <vector>    // 容器
#include <algorithm> // 算法
#include <functional> // 函数对象
using namespace std;

int main() {
    // 容器: 存储数据
    vector<int> numbers = {5, 2, 8, 1, 9, 3};

    cout << "原始数据: ";
    // 迭代器: 访问元素
    for (auto it = numbers.begin(); it != numbers.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // 算法: 对数据进行操作
    sort(numbers.begin(), numbers.end());

    cout << "排序后: ";
    for (int num : numbers) {
        cout << num << " ";
    }
    cout << endl;

    // 函数对象: 自定义操作
    auto count_even = count_if(numbers.begin(), numbers.end(),
                               [](int n) { return n % 2 == 0; });

    cout << "偶数个数: " << count_even << endl;

    return 0;
}
```

3.2 STL的设计理念

STL采用了泛型编程的思想, 通过模板实现类型无关的编程:

cpp

```
#include <iostream>
#include <vector>
#include <list>
#include <algorithm>
using namespace std;

// 通用函数，可以处理任何容器
template<typename Container>
void printContainer(const Container& container, const string& name) {
    cout << name << ": ";
    for (const auto& element : container) {
        cout << element << " ";
    }
    cout << endl;
}

int main() {
    // 相同的算法可以应用于不同的容器
    vector<int> vec = {1, 2, 3, 4, 5};
    list<int> lst = {5, 4, 3, 2, 1};

    printContainer(vec, "Vector");
    printContainer(lst, "List");

    // 相同的算法可以应用于不同的数据类型
    vector<string> words = {"hello", "world", "cpp", "stl"};
    sort(words.begin(), words.end());
    printContainer(words, "Sorted words");

    return 0;
}
```

第四部分：容器详解

4.1 容器分类概述

STL容器可以分为以下几类：

1. 序列容器：按顺序存储元素

- `vector`：动态数组
- `deque`：双端队列
- `list`：双向链表
- `array`：固定大小数组

- `forward_list`: 单向链表

2. 关联容器: 按键值存储元素

- `set`: 唯一键的集合
- `multiset`: 允许重复键的集合
- `map`: 键值对映射
- `multimap`: 允许重复键的映射

3. 容器适配器: 基于其他容器实现的特殊接口

- `stack`: 栈
- `queue`: 队列
- `priority_queue`: 优先级队列

4.2 vector详解

`vector`是最常用的STL容器，类似于动态数组。

vector基本操作

```
cpp
```

```
#include <iostream>
#include <vector>
using namespace std;

int main() {
    // 创建vector的多种方式
    vector<int> v1;           // 空vector
    vector<int> v2(5);        // 5个元素，默认值为0
    vector<int> v3(5, 10);     // 5个元素，值都是10
    vector<int> v4 = {1, 2, 3, 4, 5}; // 初始化列表
    vector<int> v5(v4);        // 复制构造

    // 基本信息
    cout << "v4的大小: " << v4.size() << endl;
    cout << "v4是否为空: " << v4.empty() << endl;
    cout << "v4的容量: " << v4.capacity() << endl;

    // 添加元素
    v1.push_back(100);
    v1.push_back(200);
    v1.push_back(300);

    // 访问元素
    cout << "v1[0] = " << v1[0] << endl;    // 不检查边界
    cout << "v1.at(1) = " << v1.at(1) << endl; // 检查边界
    cout << "v1.front() = " << v1.front() << endl; // 第一个元素
    cout << "v1.back() = " << v1.back() << endl; // 最后一个元素

    // 修改大小
    v1.resize(5, 999); // 调整为5个元素，新元素值为999

    // 遍历
    cout << "v1内容: ";
    for (size_t i = 0; i < v1.size(); ++i) {
        cout << v1[i] << " ";
    }
    cout << endl;

    // 范围for循环
    cout << "v4内容: ";
    for (const auto& element : v4) {
        cout << element << " ";
    }
    cout << endl;
}
```

```
    return 0;
}
```

vector的内存管理

```
cpp

#include <iostream>
#include <vector>
using namespace std;

void showVectorInfo(const vector<int>& v, const string& name) {
    cout << name << " - 大小: " << v.size()
        << ", 容量: " << v.capacity() << endl;
}

int main() {
    vector<int> v;
    showVectorInfo(v, "初始状态");

    // 观察容量如何增长
    for (int i = 0; i < 10; ++i) {
        v.push_back(i);
        showVectorInfo(v, "加入元素" + to_string(i) + "后");
    }

    // 预分配内存
    vector<int> v2;
    v2.reserve(100); // 预分配容量
    showVectorInfo(v2, "预分配后");

    // 释放多余内存
    v.shrink_to_fit();
    showVectorInfo(v, "收缩后");

    return 0;
}
```

vector的插入和删除

```
cpp
```

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

void printVector(const vector<int>& v, const string& msg) {
    cout << msg << ": ";
    for (int x : v) cout << x << " ";
    cout << endl;
}

int main() {
    vector<int> v = {1, 2, 3, 4, 5};
    printVector(v, "初始");

    // 在指定位置插入
    auto it = v.begin() + 2; // 指向第3个元素
    v.insert(it, 99);        // 在第3个位置插入99
    printVector(v, "插入99后");

    // 插入多个元素
    v.insert(v.begin(), 3, 0); // 在开头插入3个0
    printVector(v, "插入3个0后");

    // 插入另一个容器的元素
    vector<int> temp = {100, 200};
    v.insert(v.end(), temp.begin(), temp.end());
    printVector(v, "插入temp后");

    // 删除元素
    v.pop_back(); // 删除最后一个元素
    printVector(v, "删除最后元素后");

    // 删除指定位置的元素
    v.erase(v.begin()); // 删除第一个元素
    printVector(v, "删除第一个元素后");

    // 删除范围
    v.erase(v.begin() + 1, v.begin() + 3); // 删除第2、3个元素
    printVector(v, "删除范围后");

    // 清空
    v.clear();
    cout << "清空后大小: " << v.size() << endl;
```

```
return 0;  
}
```

4.3 list详解

list是双向链表实现，支持快速插入和删除。

cpp

```
#include <iostream>
#include <list>
#include <algorithm>
using namespace std;

void printList(const list<int>& lst, const string& msg) {
    cout << msg << ": ";
    for (int x : lst) cout << x << " ";
    cout << endl;
}
```

```
int main() {
    // 创建list
    list<int> lst1 = {1, 3, 5, 7, 9};
    list<int> lst2 = {2, 4, 6, 8};

    printList(lst1, "lst1初始");
    printList(lst2, "lst2初始");

    // list特有的操作
    lst1.push_front(0); // 在前面添加
    lst1.push_back(11); // 在后面添加
    printList(lst1, "添加元素后");

    // 排序 (list有自己的sort方法)
    lst2.sort();
    printList(lst2, "lst2排序后");

    // 合并两个已排序的list
    lst1.sort();
    printList(lst1, "lst1排序后");
    lst1.merge(lst2); // lst2会变空
    printList(lst1, "合并后");
    printList(lst2, "合并后的lst2");

    // 去重 (需要先排序)
    lst1.push_back(5);
    lst1.push_back(5);
    lst1.sort();
    printList(lst1, "添加重复元素后");
    lst1.unique(); // 去除相邻的重复元素
    printList(lst1, "去重后");

    // 移除特定值
    lst1.remove(5); // 移除所有值为5的元素
    printList(lst1, "移除5后");
}
```



```
// 移除满足条件的元素
```

```
lst1.remove_if([](int x) { return x % 2 == 0; }); // 移除偶数  
printList(lst1, "移除偶数后");
```

```
return 0;
```

```
}
```

4.4 deque详解

`deque`（双端队列）结合了vector和list的优点。

cpp

```

#include <iostream>
#include <deque>
using namespace std;

void printDeque(const deque<int>& dq, const string& msg) {
    cout << msg << ": ";
    for (int x : dq) cout << x << " ";
    cout << endl;
}

int main() {
    deque<int> dq;

    // 双端操作
    for (int i = 1; i <= 5; ++i) {
        dq.push_back(i); // 后面添加
        dq.push_front(-i); // 前面添加
    }
    printDeque(dq, "双端添加后");

    // 随机访问（像vector一样）
    cout << "中间元素: " << dq[dq.size()/2] << endl;

    // 双端删除
    dq.pop_front(); // 删除前面
    dq.pop_back(); // 删除后面
    printDeque(dq, "双端删除后");

    // 插入和删除（像vector一样）
    dq.insert(dq.begin() + 2, 999);
    printDeque(dq, "插入999后");

    dq.erase(dq.begin() + 2);
    printDeque(dq, "删除999后");

    return 0;
}

```

4.5 set和map详解

set的使用

cpp

```
#include <iostream>
#include <set>
#include <string>
using namespace std;

int main() {
    // set自动排序并去重
    set<int> s1 = {5, 2, 8, 2, 1, 9, 5}; // 重复元素会被自动去除

    cout << "set内容: ";
    for (int x : s1) cout << x << " "; // 输出: 1 2 5 8 9
    cout << endl;

    // 插入元素
    auto result = s1.insert(3);
    if (result.second) {
        cout << "成功插入3" << endl;
    }

    result = s1.insert(5); // 尝试插入已存在的元素
    if (!result.second) {
        cout << "5已存在, 插入失败" << endl;
    }

    // 查找元素
    auto it = s1.find(8);
    if (it != s1.end()) {
        cout << "找到元素: " << *it << endl;
    }

    // 删除元素
    s1.erase(2); // 按值删除
    s1.erase(s1.find(9)); // 按迭代器删除

    cout << "删除后: ";
    for (int x : s1) cout << x << " ";
    cout << endl;

    // multiset允许重复
    multiset<string> words = {"apple", "banana", "apple", "cherry"};
    cout << "multiset内容: ";
    for (const string& word : words) cout << word << " ";
    cout << endl;

    cout << "apple的数量: " << words.count("apple") << endl;
}
```

```
return 0;  
}
```

map的使用

cpp

```
#include <iostream>
#include <map>
#include <string>
using namespace std;

int main() {
    // 创建map
    map<string, int> scores;

    // 插入键值对
    scores["Alice"] = 95;
    scores["Bob"] = 87;
    scores["Charlie"] = 92;

    // 使用insert
    scores.insert({"David", 88});
    scores.insert(pair<string, int>("Eve", 91));

    // 访问元素
    cout << "Alice的分数: " << scores["Alice"] << endl;
    cout << "Bob的分数: " << scores.at("Bob") << endl; // 安全访问

    // 遍历map
    cout << "所有分数:" << endl;
    for (const auto& pair : scores) {
        cout << pair.first << ": " << pair.second << endl;
    }

    // 查找
    auto it = scores.find("Charlie");
    if (it != scores.end()) {
        cout << "找到Charlie, 分数: " << it->second << endl;
    }

    // 修改值
    scores["Alice"] += 5; // Alice加5分
    cout << "Alice新分数: " << scores["Alice"] << endl;

    // 删除
    scores.erase("Bob");

    cout << "删除Bob后的分数:" << endl;
    for (const auto& pair : scores) {
        cout << pair.first << ": " << pair.second << endl;
    }
}
```

```
return 0;  
}
```

4.6 容器适配器

stack（栈）

cpp

```
#include <iostream>
#include <stack>
#include <string>
using namespace std;

// 使用栈实现括号匹配检查
bool isValidParentheses(const string& s) {
    stack<char> stk;

    for (char c : s) {
        if (c == '(' || c == '[' || c == '{') {
            stk.push(c);
        } else if (c == ')' || c == ']' || c == '}') {
            if (stk.empty()) return false;

            char top = stk.top();
            stk.pop();

            if ((c == ')' && top != '(') ||
                (c == ']' && top != '[') ||
                (c == '}' && top != '{')) {
                return false;
            }
        }
    }

    return stk.empty();
}

int main() {
    stack<int> stk;

    // 基本操作
    stk.push(1);
    stk.push(2);
    stk.push(3);

    cout << "栈的大小: " << stk.size() << endl;

    while (!stk.empty()) {
        cout << "栈顶元素: " << stk.top() << endl;
        stk.pop();
    }

    // 实际应用: 括号匹配
    vector<string> testCases = {
```

```
    "()",
    "({})",
    "()",
    "({})",
    "{}()"
};

for (const string& test : testCases) {
    cout << "\"" << test << "\" 是否有效: "
         << (isValidParentheses(test) ? "是" : "否") << endl;
}

return 0;
}
```

queue (队列)

cpp


```

#include <iostream>
#include <queue>
#include <string>
using namespace std;

// 简单的任务调度模拟
class Task {
public:
    int id;
    string description;

    Task(int i, const string& desc) : id(i), description(desc) {}
};

int main() {
    queue<Task> taskQueue;

    // 添加任务
    taskQueue.push(Task(1, "处理邮件"));
    taskQueue.push(Task(2, "更新文档"));
    taskQueue.push(Task(3, "代码审查"));
    taskQueue.push(Task(4, "会议准备"));

    cout << "任务队列中有 " << taskQueue.size() << " 个任务" << endl;

    // 处理任务 (FIFO)
    int taskNum = 1;
    while (!taskQueue.empty()) {
        Task currentTask = taskQueue.front();
        taskQueue.pop();

        cout << "正在处理任务 " << taskNum++ << ": "
             << "ID=" << currentTask.id
             << ", 描述=" << currentTask.description << endl;
    }

    return 0;
}

```

priority_queue（优先级队列）

cpp

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

// 自定义比较函数的结构体
struct Task {
    int priority;
    string description;

    Task(int p, const string& desc) : priority(p), description(desc) {}

    // 重载比较运算符 (注意: priority_queue是大根堆, 所以这里用<)
    bool operator<(const Task& other) const {
        return priority < other.priority; // 优先级高的排在前面
    }
};

int main() {
    // 默认是大根堆 (最大元素在顶部)
    priority_queue<int> pq1;
    pq1.push(3);
    pq1.push(1);
    pq1.push(4);
    pq1.push(1);
    pq1.push(5);

    cout << "大根堆顺序: ";
    while (!pq1.empty()) {
        cout << pq1.top() << " ";
        pq1.pop();
    }
    cout << endl;

    // 小根堆 (最小元素在顶部)
    priority_queue<int, vector<int>, greater<int>> pq2;
    pq2.push(3);
    pq2.push(1);
    pq2.push(4);
    pq2.push(1);
    pq2.push(5);

    cout << "小根堆顺序: ";
    while (!pq2.empty()) {
        cout << pq2.top() << " ";
        pq2.pop();
    }
}
```

```

}
cout << endl;

// 自定义类型的优先级队列
priority_queue<Task> taskQueue;
taskQueue.push(Task(1, "普通任务"));
taskQueue.push(Task(5, "紧急任务"));
taskQueue.push(Task(3, "重要任务"));
taskQueue.push(Task(2, "次要任务"));

cout << "按优先级处理任务:" << endl;
while (!taskQueue.empty()) {
    Task t = taskQueue.top();
    taskQueue.pop();
    cout << "优先级 " << t.priority << ": " << t.description << endl;
}

return 0;
}

```

4.7 容器选择指南

cpp

```
#include <iostream>
#include <vector>
#include <list>
#include <deque>
#include <set>
#include <map>
#include <chrono>
using namespace std;
using namespace chrono;

void performanceTest() {
    const int N = 100000;

    // vector: 随机访问快, 尾部插入快
    vector<int> vec;
    auto start = high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        vec.push_back(i);
    }
    auto end = high_resolution_clock::now();
    cout << "vector尾部插入用时: "
         << duration_cast<microseconds>(end - start).count() << "微秒" << endl;

    // list: 任意位置插入删除快
    list<int> lst;
    start = high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        lst.push_back(i);
    }
    end = high_resolution_clock::now();
    cout << "list尾部插入用时: "
         << duration_cast<microseconds>(end - start).count() << "微秒" << endl;

    // deque: 两端插入删除快, 随机访问较快
    deque<int> dq;
    start = high_resolution_clock::now();
    for (int i = 0; i < N; ++i) {
        dq.push_back(i);
    }
    end = high_resolution_clock::now();
    cout << "deque尾部插入用时: "
         << duration_cast<microseconds>(end - start).count() << "微秒" << endl;
}

int main() {
    cout << "=== 容器选择指南 ===" << endl;
```

```
cout << "\n1. vector - 适用场景:" << endl;
cout << " - 需要随机访问元素" << endl;
cout << " - 主要在尾部添加/删除元素" << endl;
cout << " - 内存使用效率要求高" << endl;

cout << "\n2. list - 适用场景:" << endl;
cout << " - 频繁在中间插入/删除元素" << endl;
cout << " - 不需要随机访问" << endl;
cout << " - 元素数量变化很大" << endl;

cout << "\n3. deque - 适用场景:" << endl;
cout << " - 需要在两端插入/删除元素" << endl;
cout << " - 需要随机访问（但不如vector快）" << endl;

cout << "\n4. set/map - 适用场景:" << endl;
cout << " - 需要自动排序" << endl;
cout << " - 频繁查找元素" << endl;
cout << " - 需要唯一性约束（set）或键值对应（map）" << endl;

cout << "\n5. stack/queue/priority_queue - 适用场景:" << endl;
cout << " - 特定的访问模式需求" << endl;
cout << " - 算法设计需要特定的数据结构" << endl;

cout << "\n=== 性能测试 ===" << endl;
performanceTest();

return 0;
}
```

第五部分：迭代器深入

5.4 迭代器辅助函数

STL提供了一些有用的迭代器辅助函数：

cpp

```

#include <iostream>
#include <vector>
#include <list>
#include <iterator>
using namespace std;

int main() {
    vector<int> vec = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    cout << "=== 迭代器辅助函数 ===" << endl;

    // advance() - 移动迭代器
    auto it1 = vec.begin();
    cout << "初始位置: " << *it1 << endl;

    advance(it1, 3); // 向前移动3个位置
    cout << "advance(it1, 3)后: " << *it1 << endl;

    // distance() - 计算距离
    auto it2 = vec.begin();
    auto it3 = vec.end();
    cout << "begin到end的距离: " << distance(it2, it3) << endl;

    it2 = vec.begin() + 2;
    it3 = vec.begin() + 7;
    cout << "位置2到位置7的距离: " << distance(it2, it3) << endl;

    // next() 和 prev() (C++11)
    auto it4 = vec.begin();
    auto it5 = next(it4, 3); // 返回向前3个位置的迭代器
    auto it6 = prev(vec.end(), 2); // 返回向后2个位置的迭代器

    cout << "next(begin, 3): " << *it5 << endl;
    cout << "prev(end, 2): " << *it6 << endl;

    // 对于非随机访问迭代器也适用
    list<int> lst = {10, 20, 30, 40, 50};
    auto list_it = lst.begin();
    advance(list_it, 2);
    cout << "list中advance(begin, 2): " << *list_it << endl;

    return 0;
}

```

5.5 迭代器适配器

STL提供了几种迭代器适配器来扩展功能：

cpp

```
#include <iostream>
#include <vector>
#include <iterator>
#include <algorithm>
using namespace std;

int main() {
    cout << "=== 插入迭代器 ===" << endl;

    vector<int> vec1 = {1, 2, 3};
    vector<int> vec2;

    // back_insert_iterator - 在容器尾部插入
    copy(vec1.begin(), vec1.end(), back_inserter(vec2));

    cout << "使用back_inserter复制后vec2: ";
    for (int x : vec2) cout << x << " ";
    cout << endl;

    // front_insert_iterator - 在容器头部插入 (需要支持push_front)
    list<int> lst1 = {4, 5, 6};
    list<int> lst2;

    copy(lst1.begin(), lst1.end(), front_inserter(lst2));

    cout << "使用front_inserter复制后lst2: ";
    for (int x : lst2) cout << x << " ";
    cout << endl;

    // insert_iterator - 在指定位置插入
    vector<int> vec3 = {1, 5};
    copy(vec1.begin(), vec1.end(), inserter(vec3, vec3.begin() + 1));

    cout << "使用inserter插入后vec3: ";
    for (int x : vec3) cout << x << " ";
    cout << endl;

    cout << "\n=== 流迭代器 ===" << endl;

    // ostream_iterator - 输出流迭代器
    vector<int> numbers = {1, 2, 3, 4, 5};
    cout << "使用ostream_iterator输出: ";
    copy(numbers.begin(), numbers.end(), ostream_iterator<int>(cout, " "));
    cout << endl;

    // istream_iterator - 输入流迭代器 (需要手动输入)
```



```

cout << "请输入几个整数（输入非数字结束）：";
vector<int> input_numbers;
copy(istream_iterator<int>(cin), istream_iterator<int>(),
     back_inserter(input_numbers));

cout << "你输入的数字: ";
for (int x : input_numbers) cout << x << " ";
cout << endl;

cout << "\n=== 反向迭代器 ===" << endl;

vector<int> data = {1, 2, 3, 4, 5};

cout << "正向输出: ";
copy(data.begin(), data.end(), ostream_iterator<int>(cout, " "));
cout << endl;

cout << "反向输出: ";
copy(data.rbegin(), data.rend(), ostream_iterator<int>(cout, " "));
cout << endl;

return 0;
}

```

5.6 自定义迭代器

虽然很少需要，但了解如何创建自定义迭代器有助于理解STL：

```
cpp
```

```
#include <iostream>
#include <iterator>
using namespace std;

// 简单的整数范围迭代器
class Rangelterator {
private:
    int current;

public:
    // 迭代器所需的类型定义
    using iterator_category = forward_iterator_tag;
    using value_type = int;
    using difference_type = ptrdiff_t;
    using pointer = int*;
    using reference = int&;

    explicit Rangelterator(int start) : current(start) {}

    // 解引用操作符
    int operator*() const { return current; }

    // 前置递增
    Rangelterator& operator++() {
        ++current;
        return *this;
    }

    // 后置递增
    Rangelterator operator++(int) {
        Rangelterator temp = *this;
        ++current;
        return temp;
    }

    // 比较操作符
    bool operator==(const Rangelterator& other) const {
        return current == other.current;
    }

    bool operator!=(const Rangelterator& other) const {
        return current != other.current;
    }
};

// 范围类
```

```

class Range {
private:
    int start, end;

public:
    Range(int s, int e) : start(s), end(e) {}

    RangeIterator begin() const { return RangeIterator(start); }
    RangeIterator end() const { return RangeIterator(end); }
};

int main() {
    cout << "=== 自定义迭代器示例 ===" << endl;

    Range r(1, 6); // 范围 [1, 6)

    cout << "使用自定义迭代器遍历范围: ";
    for (auto it = r.begin(); it != r.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // 也可以使用范围for循环
    cout << "使用范围for循环: ";
    for (int value : r) {
        cout << value << " ";
    }
    cout << endl;

    // 使用STL算法
    cout << "使用STL算法: ";
    copy(r.begin(), r.end(), ostream_iterator<int>(cout, " "));
    cout << endl;

    return 0;
}

```

5.7 迭代器失效

理解迭代器失效对于编写正确的STL代码至关重要：

cpp

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

void demonstrateIteratorInvalidation() {
    cout << "=== 迭代器失效演示 ===" << endl;

    // vector的迭代器失效
    cout << "1. vector迭代器失效:" << endl;
    vector<int> vec = {1, 2, 3, 4, 5};

    auto it = vec.begin() + 2; // 指向元素3
    cout << "插入前it指向: " << *it << endl;

    vec.insert(vec.begin(), 0); // 在开头插入, 可能导致重新分配
    // it现在可能失效!
    // cout << "插入后it指向: " << *it << endl; // 危险!

    cout << "重新获取迭代器" << endl;
    it = vec.begin() + 3; // 重新定位到元素3
    cout << "新it指向: " << *it << endl;

    // list的迭代器更稳定
    cout << "\n2. list迭代器稳定性:" << endl;
    list<int> lst = {1, 2, 3, 4, 5};

    auto list_it = lst.begin();
    advance(list_it, 2); // 指向元素3
    cout << "插入前list_it指向: " << *list_it << endl;

    lst.insert(lst.begin(), 0); // 在开头插入
    cout << "插入后list_it仍指向: " << *list_it << endl; // 仍然有效

    // 删除操作的影响
    cout << "\n3. 删除操作对迭代器的影响:" << endl;
    vector<int> vec2 = {1, 2, 3, 4, 5};

    auto it1 = vec2.begin() + 1; // 指向元素2
    auto it2 = vec2.begin() + 3; // 指向元素4

    cout << "删除前: it1=" << *it1 << ", it2=" << *it2 << endl;

    vec2.erase(vec2.begin() + 2); // 删除元素3

    cout << "删除后: it1=" << *it1 << endl; // it1仍有效
```

```
// it2可能失效，因为它指向被删除元素之后的位置
```

```
// 安全的删除方式
```

```
cout << "\n4. 安全的删除方式:" << endl;
```

```
vector<int> vec3 = {1, 2, 3, 4, 5, 2, 6, 2};
```

```
// 删除所有值为2的元素
```

```
auto iter = vec3.begin();
```

```
while (iter != vec3.end()) {
```

```
    if (*iter == 2) {
```

```
        iter = vec3.erase(iter); // erase返回下一个有效迭代器
```

```
    } else {
```

```
        ++iter;
```

```
    }
```

```
}
```

```
cout << "删除所有2后的vec3: ";
```

```
for (int x : vec3) cout << x << " ";
```

```
cout << endl;
```

```
}
```

```
int main() {
```

```
    demonstratIteratorInvalidation();
```

```
    return 0;
```

```
}
```

第六部分：算法详解

6.1 算法分类概述

STL算法可以分为以下几类：

1. **非修改性算法**：不改变元素值
2. **修改性算法**：可能改变元素值或顺序
3. **排序算法**：排序和相关操作
4. **数值算法**：数学运算

6.2 查找算法

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

int main() {
    vector<int> numbers = {1, 3, 5, 7, 9, 11, 13, 15};
    vector<string> words = {"apple", "banana", "cherry", "date", "elderberry"};

    cout << "=== 查找算法 ===" << endl;

    // find() - 查找值
    auto it = find(numbers.begin(), numbers.end(), 7);
    if (it != numbers.end()) {
        cout << "找到7, 位置: " << distance(numbers.begin(), it) << endl;
    }

    // find_if() - 按条件查找
    auto it2 = find_if(numbers.begin(), numbers.end(),
        [](int n) { return n > 10; });
    if (it2 != numbers.end()) {
        cout << "第一个大于10的数: " << *it2 << endl;
    }

    // find_if_not() - 查找不满足条件的元素
    auto it3 = find_if_not(numbers.begin(), numbers.end(),
        [](int n) { return n % 2 == 1; });
    if (it3 != numbers.end()) {
        cout << "第一个偶数: " << *it3 << endl;
    } else {
        cout << "所有数都是奇数" << endl;
    }

    // search() - 查找子序列
    vector<int> pattern = {7, 9};
    auto it4 = search(numbers.begin(), numbers.end(),
        pattern.begin(), pattern.end());
    if (it4 != numbers.end()) {
        cout << "找到模式{7,9}, 起始位置: "
            << distance(numbers.begin(), it4) << endl;
    }

    // binary_search() - 二分查找 (需要已排序)
    if (binary_search(numbers.begin(), numbers.end(), 9)) {
        cout << "9存在于已排序的数组中" << endl;
    }
}
```

```

}

// lower_bound() 和 upper_bound()
auto lower = lower_bound(numbers.begin(), numbers.end(), 7);
auto upper = upper_bound(numbers.begin(), numbers.end(), 7);

cout << "7的范围: [" << distance(numbers.begin(), lower)
    << ", " << distance(numbers.begin(), upper) << "]" << endl;

// equal_range() - 获取相等元素的范围
auto range = equal_range(numbers.begin(), numbers.end(), 7);
cout << "7的范围 (使用equal_range) : ["
    << distance(numbers.begin(), range.first) << ", "
    << distance(numbers.begin(), range.second) << "]" << endl;

return 0;
}

```

6.3 计数和比较算法

cpp

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <numeric>
using namespace std;

int main() {
    vector<int> vec1 = {1, 2, 3, 2, 4, 2, 5};
    vector<int> vec2 = {1, 2, 3, 4, 5};
    vector<int> vec3 = {1, 2, 3, 4, 5};

    cout << "=== 计数算法 ===" << endl;

    // count() - 计数特定值
    int count_2 = count(vec1.begin(), vec1.end(), 2);
    cout << "vec1中2的个数: " << count_2 << endl;

    // count_if() - 按条件计数
    int count_even = count_if(vec1.begin(), vec1.end(),
        [](int n) { return n % 2 == 0; });
    cout << "vec1中偶数的个数: " << count_even << endl;

    cout << "\n=== 比较算法 ===" << endl;

    // equal() - 比较两个范围是否相等
    if (equal(vec2.begin(), vec2.end(), vec3.begin())) {
        cout << "vec2和vec3相等" << endl;
    }

    // mismatch() - 找到第一个不匹配的位置
    auto mismatch_pair = mismatch(vec2.begin(), vec2.end(), vec1.begin());
    if (mismatch_pair.first != vec2.end()) {
        cout << "vec2和vec1第一个不匹配位置: "
            << distance(vec2.begin(), mismatch_pair.first)
            << ", 值分别是: " << *mismatch_pair.first
            << " 和 " << *mismatch_pair.second << endl;
    }

    // lexicographical_compare() - 字典序比较
    vector<int> smaller = {1, 2, 2};
    vector<int> larger = {1, 2, 3};

    if (lexicographical_compare(smaller.begin(), smaller.end(),
        larger.begin(), larger.end())) {
        cout << "{1,2,2} 在字典序上小于 {1,2,3}" << endl;
    }
}

```



```

cout << "\n=== 最值算法 ===" << endl;

// min_element() 和 max_element()
auto min_it = min_element(vec1.begin(), vec1.end());
auto max_it = max_element(vec1.begin(), vec1.end());

cout << "vec1最小值: " << *min_it
    << ", 位置: " << distance(vec1.begin(), min_it) << endl;
cout << "vec1最大值: " << *max_it
    << ", 位置: " << distance(vec1.begin(), max_it) << endl;

// minmax_element() - 同时找最小和最大值
auto minmax_pair = minmax_element(vec1.begin(), vec1.end());
cout << "最小值: " << *minmax_pair.first
    << ", 最大值: " << *minmax_pair.second << endl;

return 0;
}

```

6.4 修改性算法

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <iterator>
using namespace std;

void printVector(const vector<int>& vec, const string& name) {
    cout << name << ": ";
    for (int x : vec) cout << x << " ";
    cout << endl;
}

int main() {
    cout << "=== 复制算法 ===" << endl;

    vector<int> source = {1, 2, 3, 4, 5};
    vector<int> dest(5);

    // copy() - 复制
    copy(source.begin(), source.end(), dest.begin());
    printVector(dest, "复制后的dest");

    // copy_if() - 按条件复制
    vector<int> evens;
    copy_if(source.begin(), source.end(), back_inserter(evens),
        [](int n) { return n % 2 == 0; });
    printVector(evens, "偶数");

    // copy_n() - 复制n个元素
    vector<int> first_three(3);
    copy_n(source.begin(), 3, first_three.begin());
    printVector(first_three, "前三个");

    cout << "\n=== 变换算法 ===" << endl;

    // transform() - 单元操作
    vector<int> squares(source.size());
    transform(source.begin(), source.end(), squares.begin(),
        [](int n) { return n * n; });
    printVector(squares, "平方");

    // transform() - 二元操作
    vector<int> vec1 = {1, 2, 3, 4, 5};
    vector<int> vec2 = {2, 3, 4, 5, 6};
    vector<int> sums(vec1.size());
```

```
transform(vec1.begin(), vec1.end(), vec2.begin(), sums.begin(),
    [](int a, int b) { return a + b; });
printVector(sums, "对应元素相加");

cout << "\n=== 填充算法 ===" << endl;

// fill() - 填充值
vector<int> filled(5);
fill(filled.begin(), filled.end(), 42);
printVector(filled, "填充42");

// fill_n() - 填充n个值
fill_n(filled.begin(), 3, 99);
printVector(filled, "前3个填充99");

// generate() - 用函数生成值
vector<int> generated(5);
int counter = 0;
generate(generated.begin(), generated.end(),
    [&counter]() { return ++counter * 10; });
printVector(generated, "生成的值");

cout << "\n=== 替换算法 ===" << endl;

vector<int> data = {1, 2, 3, 2, 4, 2, 5};
printVector(data, "原始数据");

// replace() - 替换值
replace(data.begin(), data.end(), 2, 99);
printVector(data, "替换2为99");

// replace_if() - 按条件替换
replace_if(data.begin(), data.end(),
    [](int n) { return n > 50; }, 0);
printVector(data, "大于50的替换为0");

// replace_copy() - 复制并替换
vector<int> original = {1, 2, 3, 2, 4};
vector<int> replaced(original.size());
replace_copy(original.begin(), original.end(),
    replaced.begin(), 2, 888);
printVector(original, "原始");
printVector(replaced, "复制并替换2为888");

return 0;
}
```

6.5 排序算法

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
#include <functional>
using namespace std;

void printVector(const vector<int>& vec, const string& name) {
    cout << name << ": ";
    for (int x : vec) cout << x << " ";
    cout << endl;
}

int main() {
    cout << "=== 排序算法 ===" << endl;

    vector<int> data = {5, 2, 8, 1, 9, 3, 7, 4, 6};
    printVector(data, "原始数据");

    // sort() - 快速排序
    vector<int> sorted_data = data;
    sort(sorted_data.begin(), sorted_data.end());
    printVector(sorted_data, "升序排序");

    // 降序排序
    sort(sorted_data.begin(), sorted_data.end(), greater<int>());
    printVector(sorted_data, "降序排序");

    // 自定义比较函数
    vector<int> custom_sorted = data;
    sort(custom_sorted.begin(), custom_sorted.end(),
        [](int a, int b) { return a % 10 < b % 10; }); // 按个位数排序
    printVector(custom_sorted, "按个位数排序");

    cout << "\n=== 稳定排序 ===" << endl;

    // stable_sort() - 稳定排序
    struct Person {
        string name;
        int age;
        Person(string n, int a) : name(n), age(a) {}
    };

    vector<Person> people = {
        {"Alice", 25}, {"Bob", 30}, {"Charlie", 25}, {"David", 30}
    };
};
```

// 按年龄稳定排序

```
stable_sort(people.begin(), people.end(),
    [](const Person& a, const Person& b) {
        return a.age < b.age;
    });
```

```
cout << "按年龄稳定排序:" << endl;
```

```
for (const auto& p : people) {
    cout << p.name << "(" << p.age << ") ";
}
cout << endl;
```

```
cout << "\n=== 部分排序 ===" << endl;
```

```
vector<int> large_data = {9, 5, 3, 8, 2, 7, 1, 6, 4};
printVector(large_data, "原始");
```

// partial_sort() - 部分排序，只排序前n个元素

```
partial_sort(large_data.begin(), large_data.begin() + 3, large_data.end());
printVector(large_data, "部分排序（前3个）");
```

```
cout << "\n=== 第n个元素 ===" << endl;
```

```
vector<int> nth_data = {9, 5, 3, 8, 2, 7, 1, 6, 4};
```

// nth_element() - 使第n个位置的元素处于正确位置

```
nth_element(nth_data.begin(), nth_data.begin() + 4, nth_data.end());
cout << "第5小的元素: " << nth_data[4] << endl;
printVector(nth_data, "nth_element后");
```

```
cout << "\n=== 其他排序相关算法 ===" << endl;
```

// is_sorted() - 检查是否已排序

```
vector<int> sorted_check = {1, 2, 3, 4, 5};
vector<int> unsorted_check = {1, 3, 2, 4, 5};
```

```
cout << "{1,2,3,4,5} 是否已排序: " << is_sorted(sorted_check.begin(), sorted_check.end()) << endl;
cout << "{1,3,2,4,5} 是否已排序: " << is_sorted(unsorted_check.begin(), unsorted_check.end()) << endl;
```

// is_sorted_until() - 找到第一个未排序的位置

```
auto it = is_sorted_until(unsorted_check.begin(), unsorted_check.end());
cout << "第一个未排序位置的值: " << *it << endl;
```

```
return 0;
```

```
}
```

6.6 排列和洗牌算法

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>
using namespace std;

int main() {
    cout << "=== 排列算法 ===" << endl;

    vector<int> data = {1, 2, 3};

    cout << "所有排列:" << endl;
    do {
        for (int x : data) cout << x << " ";
        cout << endl;
    } while (next_permutation(data.begin(), data.end()));

    cout << "\n反向排列:" << endl;
    data = {3, 2, 1}; // 从最大排列开始
    do {
        for (int x : data) cout << x << " ";
        cout << endl;
    } while (prev_permutation(data.begin(), data.end()));

    cout << "\n=== 洗牌算法 ===" << endl;

    vector<int> deck = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    cout << "原始: ";
    for (int x : deck) cout << x << " ";
    cout << endl;

    // 使用随机数生成器
    random_device rd;
    mt19937 g(rd());

    // shuffle() - 随机洗牌
    shuffle(deck.begin(), deck.end(), g);

    cout << "洗牌后: ";
    for (int x : deck) cout << x << " ";
    cout << endl;

    cout << "\n=== 旋转算法 ===" << endl;

    vector<int> rotate_data = {1, 2, 3, 4, 5, 6, 7};
```



```

cout << "原始: ";
for (int x : rotate_data) cout << x << " ";
cout << endl;

// rotate() - 旋转元素
rotate(rotate_data.begin(), rotate_data.begin() + 3, rotate_data.end());

cout << "左旋转3位: ";
for (int x : rotate_data) cout << x << " ";
cout << endl;

cout << "\n=== 反转算法 ===" << endl;

vector<int> reverse_data = {1, 2, 3, 4, 5};

cout << "原始: ";
for (int x : reverse_data) cout << x << " ";
cout << endl;

// reverse() - 反转
reverse(reverse_data.begin(), reverse_data.end());

cout << "反转后: ";
for (int x : reverse_data) cout << x << " ";
cout << endl;

return 0;
}

```

6.7 数值算法

cpp

```
#include <iostream>
#include <vector>
#include <numeric>
#include <functional>
using namespace std;

int main() {
    vector<int> numbers = {1, 2, 3, 4, 5};

    cout << "=== 累积算法 ===" << endl;

    // accumulate() - 累积
    int sum = accumulate(numbers.begin(), numbers.end(), 0);
    cout << "求和: " << sum << endl;

    // 使用不同的操作
    int product = accumulate(numbers.begin(), numbers.end(), 1, multiplies<int>());
    cout << "求积: " << product << endl;

    // 自定义操作
    int sum_of_squares = accumulate(numbers.begin(), numbers.end(), 0,
        [](int acc, int val) { return acc + val * val; });
    cout << "平方和: " << sum_of_squares << endl;

    cout << "\n=== 内积算法 ===" << endl;

    vector<int> vec1 = {1, 2, 3, 4};
    vector<int> vec2 = {2, 3, 4, 5};

    // inner_product() - 内积
    int inner_prod = inner_product(vec1.begin(), vec1.end(), vec2.begin(), 0);
    cout << "内积: " << inner_prod << endl; // 1*2 + 2*3 + 3*4 + 4*5 = 40

    cout << "\n=== 部分和算法 ===" << endl;

    vector<int> input = {1, 2, 3, 4, 5};
    vector<int> partial_sums(input.size());

    // partial_sum() - 部分和
    partial_sum(input.begin(), input.end(), partial_sums.begin());

    cout << "原始: ";
    for (int x : input) cout << x << " ";
    cout << endl;

    cout << "部分和: ";
```

```

for (int x : partial_sums) cout << x << " ";
cout << endl;

cout << "\n=== 相邻差算法 ===" << endl;

vector<int> differences(input.size());

// adjacent_difference() - 相邻差
adjacent_difference(partial_sums.begin(), partial_sums.end(),
                    differences.begin());

cout << "部分和: ";
for (int x : partial_sums) cout << x << " ";
cout << endl;

cout << "相邻差: ";
for (int x : differences) cout << x << " ";
cout << endl;

cout << "\n=== 序列生成算法 ===" << endl;

vector<int> sequence(10);

// iota() - 生成递增序列
iota(sequence.begin(), sequence.end(), 1); // 从1开始

cout << "递增序列: ";
for (int x : sequence) cout << x << " ";
cout << endl;

return 0;
}

```

第七部分：函数对象

7.1 函数对象基础

函数对象（函数符）是重载了的类对象，可以像函数一样调用：`operator()`

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

// 自定义函数对象
class Square {
public:
    int operator()(int x) const {
        return x * x;
    }
};

// 带状态的函数对象
class Counter {
private:
    int count;

public:
    Counter() : count(0) {}

    int operator()() {
        return ++count;
    }

    int getCount() const { return count; }
};

// 谓词函数对象
class IsEven {
public:
    bool operator()(int x) const {
        return x % 2 == 0;
    }
};

int main() {
    cout << "=== 基本函数对象 ===" << endl;

    Square square;
    cout << "5的平方: " << square(5) << endl;

    Counter counter;
    cout << "计数器: " << counter() << " " << counter() << " " << counter() << endl;
    cout << "总计数: " << counter.getCount() << endl;
```

```

cout << "\n=== 在STL算法中使用函数对象 ===" << endl;

vector<int> numbers = {1, 2, 3, 4, 5};
vector<int> squares(numbers.size());

// 使用函数对象进行变换
transform(numbers.begin(), numbers.end(), squares.begin(), Square());

cout << "原数组: ";
for (int x : numbers) cout << x << " ";
cout << endl;

cout << "平方后: ";
for (int x : squares) cout << x << " ";
cout << endl;

// 使用谓词函数对象
int even_count = count_if(numbers.begin(), numbers.end(), IsEven());
cout << "偶数个数: " << even_count << endl;

return 0;
}

```

7.2 Lambda表达式

C++11引入的Lambda表达式提供了更简洁的函数对象定义方式：

```
cpp
```

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

int main() {
    vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    cout << "=== Lambda表达式基础 ===" << endl;

    // 基本Lambda表达式
    auto square = [](int x) { return x * x; };
    cout << "5的平方: " << square(5) << endl;

    // 直接在算法中使用
    vector<int> squares(numbers.size());
    transform(numbers.begin(), numbers.end(), squares.begin(),
        [](int x) { return x * x; });

    cout << "平方: ";
    for (int x : squares) cout << x << " ";
    cout << endl;

    cout << "\n=== 捕获变量 ===" << endl;

    int multiplier = 3;

    // 按值捕获
    auto multiply_by_value = [multiplier](int x) { return x * multiplier; };

    // 按引用捕获
    auto multiply_by_ref = [&multiplier](int x) { return x * multiplier; };

    // 捕获所有变量
    auto multiply_all_by_value = [=](int x) { return x * multiplier; };
    auto multiply_all_by_ref = [&](int x) { return x * multiplier; };

    cout << "按值捕获: " << multiply_by_value(5) << endl;
    cout << "按引用捕获: " << multiply_by_ref(5) << endl;

    multiplier = 4; // 修改原变量
    cout << "修改后按值捕获: " << multiply_by_value(5) << endl; // 仍然是3*5=15
    cout << "修改后按引用捕获: " << multiply_by_ref(5) << endl; // 4*5=20

    cout << "\n=== 可变Lambda ===" << endl;
```

```

int counter = 0;
auto incremter = [counter]() mutable { return ++counter; };

cout << "第一次调用: " << incremter() << endl;
cout << "第二次调用: " << incremter() << endl;
cout << "原变量counter: " << counter << endl; // 仍然是0

cout << "\n=== 泛型Lambda (C++14) ===" << endl;

// 使用auto参数
auto generic_adder = [](auto a, auto b) { return a + b; };

cout << "整数相加: " << generic_adder(3, 4) << endl;
cout << "浮点相加: " << generic_adder(3.5, 4.2) << endl;
cout << "字符串相加: " << generic_adder(string("Hello "), string("World")) << endl;

cout << "\n=== 在STL算法中的应用 ===" << endl;

// 查找第一个大于5的元素
auto it = find_if(numbers.begin(), numbers.end(),
    [](int x) { return x > 5; });
if (it != numbers.end()) {
    cout << "第一个大于5的元素: " << *it << endl;
}

// 计算偶数个数
int even_count = count_if(numbers.begin(), numbers.end(),
    [](int x) { return x % 2 == 0; });
cout << "偶数个数: " << even_count << endl;

// 自定义排序
vector<string> words = {"apple", "pie", "a", "elephant", "cat"};
sort(words.begin(), words.end(),
    [](const string& a, const string& b) {
        return a.length() < b.length(); // 按长度排序
    });

cout << "按长度排序: ";
for (const string& word : words) cout << word << " ";
cout << endl;

return 0;
}

```

7.3 标准函数对象

STL提供了许多预定义的函数对象：

cpp


```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

int main() {
    vector<int> numbers = {5, 2, 8, 1, 9, 3};

    cout << "=== 算术函数对象 ===" << endl;

    // plus, minus, multiplies, divides, modulus, negate
    vector<int> vec1 = {1, 2, 3, 4, 5};
    vector<int> vec2 = {2, 3, 4, 5, 6};
    vector<int> result(vec1.size());

    // 对应元素相加
    transform(vec1.begin(), vec1.end(), vec2.begin(), result.begin(),
        plus<int>());

    cout << "vec1: ";
    for (int x : vec1) cout << x << " ";
    cout << endl;

    cout << "vec2: ";
    for (int x : vec2) cout << x << " ";
    cout << endl;

    cout << "相加: ";
    for (int x : result) cout << x << " ";
    cout << endl;

    // 取负数
    transform(vec1.begin(), vec1.end(), result.begin(), negate<int>());
    cout << "取负: ";
    for (int x : result) cout << x << " ";
    cout << endl;

    cout << "\n=== 比较函数对象 ===" << endl;

    // equal_to, not_equal_to, greater, less, greater_equal, less_equal
    vector<int> unsorted = numbers;

    // 升序排序
    sort(unsorted.begin(), unsorted.end(), less<int>());
    cout << "升序: ";
```

```

for (int x : unsorted) cout << x << " ";
cout << endl;

// 降序排序
sort(unsorted.begin(), unsorted.end(), greater<int>());
cout << "降序: ";
for (int x : unsorted) cout << x << " ";
cout << endl;

cout << "\n=== 逻辑函数对象 ===" << endl;

vector<bool> bool_vec1 = {true, false, true, false};
vector<bool> bool_vec2 = {true, true, false, false};
vector<bool> bool_result(bool_vec1.size());

// 逻辑与
transform(bool_vec1.begin(), bool_vec1.end(),
          bool_vec2.begin(), bool_result.begin(),
          logical_and<bool>());

cout << "逻辑与结果: ";
for (bool b : bool_result) cout << b << " ";
cout << endl;

// 逻辑非
transform(bool_vec1.begin(), bool_vec1.end(), bool_result.begin(),
          logical_not<bool>());

cout << "逻辑非结果: ";
for (bool b : bool_result) cout << b << " ";
cout << endl;

return 0;
}

```

7.4 函数适配器

函数适配器用于修改或组合现有的函数对象：

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <functional>
using namespace std;

int main() {
    vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

    cout << "=== bind适配器 ===" << endl;

    // bind将函数和参数绑定
    auto multiply_by_2 = bind(multiplies<int>(), placeholders::_1, 2);
    auto add_10 = bind(plus<int>(), placeholders::_1, 10);

    vector<int> doubled(numbers.size());
    vector<int> added(numbers.size());

    transform(numbers.begin(), numbers.end(), doubled.begin(), multiply_by_2);
    transform(numbers.begin(), numbers.end(), added.begin(), add_10);

    cout << "原数组: ";
    for (int x : numbers) cout << x << " ";
    cout << endl;

    cout << "乘以2: ";
    for (int x : doubled) cout << x << " ";
    cout << endl;

    cout << "加10: ";
    for (int x : added) cout << x << " ";
    cout << endl;

    // 绑定成员函数
    vector<string> words = {"hello", "world", "cpp", "programming"};

    // 查找长度大于4的第一个字符串
    auto it = find_if(words.begin(), words.end(),
        bind(greater<size_t>(),
            bind(&string::length, placeholders::_1, 4)));

    if (it != words.end()) {
        cout << "第一个长度大于4的字符串: " << *it << endl;
    }

    cout << "\n=== function包装器 ===" << endl;
```

```

// function可以包装任何可调用对象
function<int(int)> func;

// 包装普通函数
func = [](int x) { return x * x; };
cout << "Lambda平方: " << func(5) << endl;

// 包装bind结果
func = bind(multiplies<int>(), placeholders::_1, 3);
cout << "绑定乘法: " << func(7) << endl;

// 包装函数对象
func = negate<int>();
cout << "取负: " << func(42) << endl;

cout << "\n=== 实际应用示例 ===" << endl;

// 复合操作: 找到第一个平方大于50的数
auto square_greater_than_50 = [](int x) { return x * x > 50; };

auto it2 = find_if(numbers.begin(), numbers.end(), square_greater_than_50);
if (it2 != numbers.end()) {
    cout << "第一个平方大于50的数: " << *it2
        << " (平方值: " << (*it2) * (*it2) << ")" << endl;
}

// 使用bind实现相同功能
auto it3 = find_if(numbers.begin(), numbers.end(),
    bind(greater<int>(),
        bind([](int x) { return x * x; }, placeholders::_1,
            50)));

if (it3 != numbers.end()) {
    cout << "使用bind找到: " << *it3 << endl;
}

return 0;
}

```

7.5 自定义函数对象的高级应用

cpp

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <map>
#include <string>
using namespace std;

// 累积统计的函数对象
class Accumulator {
private:
    int sum;
    int count;

public:
    Accumulator() : sum(0), count(0) {}

    void operator()(int value) {
        sum += value;
        count++;
    }

    double average() const {
        return count > 0 ? static_cast<double>(sum) / count : 0.0;
    }

    int getSum() const { return sum; }
    int getCount() const { return count; }
};

// 范围检查器
class RangeChecker {
private:
    int min_val, max_val;

public:
    RangeChecker(int min_v, int max_v) : min_val(min_v), max_val(max_v) {}

    bool operator()(int value) const {
        return value >= min_val && value <= max_val;
    }
};

// 多条件排序器
class PersonSorter {
public:
    struct Person {
```

```

    string name;
    int age;
    double salary;

    Person(const string& n, int a, double s)
        : name(n), age(a), salary(s) {}
};

enum SortBy { BY_NAME, BY_AGE, BY_SALARY };

private:
    SortBy criteria;
    bool ascending;

public:
    PersonSorter(SortBy c, bool asc = true) : criteria(c), ascending(asc) {}

    bool operator()(const Person& a, const Person& b) const {
        bool result;
        switch (criteria) {
            case BY_NAME:
                result = a.name < b.name;
                break;
            case BY_AGE:
                result = a.age < b.age;
                break;
            case BY_SALARY:
                result = a.salary < b.salary;
                break;
        }
        return ascending ? result : !result;
    }
};

int main() {
    cout << "=== 累积统计示例 ===" << endl;

    vector<int> scores = {85, 92, 78, 96, 88, 73, 89, 94};

    Accumulator acc = for_each(scores.begin(), scores.end(), Accumulator());

    cout << "分数: ";
    for (int score : scores) cout << score << " ";
    cout << endl;

    cout << "总分: " << acc.getSum() << endl;
    cout << "平均分: " << acc.average() << endl;
}

```

```
cout << "数量: " << acc.getCount() << endl;

cout << "\n=== 范围过滤示例 ===" << endl;

vector<int> numbers = {15, 25, 35, 45, 55, 65, 75, 85, 95};

// 统计30-70范围内的数字
RangeChecker range_30_70(30, 70);
int count_in_range = count_if(numbers.begin(), numbers.end(), range_30_70);

cout << "数字: ";
for (int num : numbers) cout << num << " ";
cout << endl;

cout << "30-70范围内的数字个数: " << count_in_range << endl;

// 过滤出范围内的数字
vector<int> filtered;
copy_if(numbers.begin(), numbers.end(), back_inserter(filtered), range_30_70);

cout << "30-70范围内的数字: ";
for (int num : filtered) cout << num << " ";
cout << endl;

cout << "\n=== 多条件排序示例 ===" << endl;

vector<PersonSorter::Person> people = {
    {"Alice", 30, 50000},
    {"Bob", 25, 60000},
    {"Charlie", 35, 55000},
    {"David", 25, 45000},
    {"Eve", 30, 65000}
};

auto printPeople = [](const vector<PersonSorter::Person>& p, const string& title) {
    cout << title << ":" << endl;
    for (const auto& person : p) {
        cout << " " << person.name << " (年龄:" << person.age
            << ", 薪资:" << person.salary << ")" << endl;
    }
};

printPeople(people, "原始数据");

// 按姓名排序
sort(people.begin(), people.end(), PersonSorter(PersonSorter::BY_NAME));
printPeople(people, "按姓名排序");
```

```
// 按年龄降序排序
sort(people.begin(), people.end(), PersonSorter(PersonSorter::BY_AGE, false));
printPeople(people, "按年龄降序排序");

// 按薪资升序排序
sort(people.begin(), people.end(), PersonSorter(PersonSorter::BY_SALARY));
printPeople(people, "按薪资升序排序");

return 0;
}
```

第八部分：实战练习

8.1 文本处理系统

创建一个完整的文本处理系统，展示STL的综合应用：

cpp


```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>
#include <map>
#include <set>
#include <algorithm>
#include <sstream>
#include <iomanip>
using namespace std;

class TextProcessor {
private:
    vector<string> lines;
    map<string, int> wordCount;
    set<string> uniqueWords;

    // 清理单词（去除标点符号，转换为小写）
    string cleanWord(const string& word) {
        string cleaned;
        for (char c : word) {
            if (isalnum(c)) {
                cleaned += tolower(c);
            }
        }
        return cleaned;
    }

public:
    // 从文件读取文本
    bool loadFromFile(const string& filename) {
        ifstream file(filename);
        if (!file.is_open()) {
            cerr << "无法打开文件: " << filename << endl;
            return false;
        }

        string line;
        while (getline(file, line)) {
            lines.push_back(line);
        }

        cout << "成功读取 " << lines.size() << " 行文本" << endl;
        return true;
    }
}
```

```
// 从用户输入读取文本
```

```
void loadFromInput() {  
    cout << "请输入文本（输入空行结束）：" << endl;  
    string line;  
    while (getline(cin, line) && !line.empty()) {  
        lines.push_back(line);  
    }  
}
```

```
// 处理文本，统计单词
```

```
void processText() {  
    wordCount.clear();  
    uniqueWords.clear();  
  
    for (const string& line : lines) {  
        istringstream iss(line);  
        string word;  
  
        while (iss >> word) {  
            string cleaned = cleanWord(word);  
            if (!cleaned.empty()) {  
                wordCount[cleaned]++;  
                uniqueWords.insert(cleaned);  
            }  
        }  
    }  
  
    cout << "文本处理完成，共发现 " << uniqueWords.size() << " 个不同单词" << endl;  
}
```

```
// 显示统计信息
```

```
void showStatistics() {  
    cout << "\n=== 文本统计 ===" << endl;  
    cout << "总行数: " << lines.size() << endl;  
    cout << "总单词数: " << accumulate(wordCount.begin(), wordCount.end(), 0,  
        [](int sum, const pair<string, int>& p) {  
            return sum + p.second;  
        }) << endl;  
    cout << "不同单词数: " << uniqueWords.size() << endl;
```

```
// 计算平均单词长度
```

```
double avgLength = accumulate(uniqueWords.begin(), uniqueWords.end(), 0.0,  
    [](double sum, const string& word) {  
        return sum + word.length();  
    }) / uniqueWords.size();  
cout << "平均单词长度: " << fixed << setprecision(2) << avgLength << endl;  
}
```

// 显示最常见的单词

```
void showTopWords(int count = 10) {
    cout << "\n=== 最常见的 " << count << " 个单词 ===" << endl;

    // 将map转换为vector进行排序
    vector<pair<string, int>> wordList(wordCount.begin(), wordCount.end());

    // 按出现次数降序排序
    sort(wordList.begin(), wordList.end(),
        [](const pair<string, int>& a, const pair<string, int>& b) {
            return a.second > b.second;
        });

    // 显示前count个
    int displayCount = min(count, static_cast<int>(wordList.size()));
    for (int i = 0; i < displayCount; ++i) {
        cout << setw(3) << (i + 1) << ". "
            << setw(15) << left << wordList[i].first
            << ": " << wordList[i].second << " 次" << endl;
    }
}
```

// 查找包含特定单词的行

```
void findLinesWithWord(const string& searchWord) {
    string cleanedSearch = cleanWord(searchWord);
    vector<int> foundLines;

    for (int i = 0; i < lines.size(); ++i) {
        string line = lines[i];
        transform(line.begin(), line.end(), line.begin(), ::tolower);

        if (line.find(cleanedSearch) != string::npos) {
            foundLines.push_back(i + 1); // 行号从1开始
        }
    }

    cout << "\n包含单词 \"" << searchWord << "\" 的行:" << endl;
    if (foundLines.empty()) {
        cout << "未找到包含该单词的行" << endl;
    } else {
        for (int lineNum : foundLines) {
            cout << "第 " << lineNum << " 行:" << lines[lineNum - 1] << endl;
        }
    }
}
```

// 查找长单词

```
void findLongWords(int minLength = 8) {  
    cout << "\n=== 长度大于等于 " << minLength << " 的单词 ===" << endl;  
  
    vector<string> longWords;  
    copy_if(uniqueWords.begin(), uniqueWords.end(),  
            back_inserter(longWords),  
            [minLength](const string& word) {  
                return word.length() >= minLength;  
            });  
}
```

// 按长度排序

```
sort(longWords.begin(), longWords.end(),  
     [](const string& a, const string& b) {  
         return a.length() > b.length();  
     });
```

```
if (longWords.empty()) {  
    cout << "没有找到长度大于等于 " << minLength << " 的单词" << endl;  
} else {  
    for (const string& word : longWords) {  
        cout << word << " (" << word.length() << " 字符，出现 "  
            << wordCount[word] << " 次)" << endl;  
    }  
}
```

// 生成单词云数据

```
void generateWordCloud() {  
    cout << "\n=== 单词云数据 ===" << endl;
```

// 获取最高频率用于缩放

```
int maxFreq = max_element(wordCount.begin(), wordCount.end(),  
                           [](const pair<string, int>& a, const pair<string, int>& b) {  
                               return a.second < b.second;  
                           })->second;
```

// 只显示出现次数大于1的单词

```
vector<pair<string, int>> cloudWords;  
copy_if(wordCount.begin(), wordCount.end(),  
        back_inserter(cloudWords),  
        [](const pair<string, int>& p) { return p.second > 1; });
```

// 按频率排序

```
sort(cloudWords.begin(), cloudWords.end(),  
     [](const pair<string, int>& a, const pair<string, int>& b) {  
         return a.second > b.second;
```

```

});

for (const auto& word : cloudWords) {
    int size = (word.second * 20) / maxFreq + 5; // 字体大小5-25
    cout << word.first << ":" << size << " ";
}
cout << endl;
}
};

```

// 演示程序

```

int main() {
    TextProcessor processor;

    cout << "=== 文本处理系统 ===" << endl;
    cout << "1. 从文件读取" << endl;
    cout << "2. 手动输入" << endl;
    cout << "选择输入方式 (1/2): ";

    int choice;
    cin >> choice;
    cin.ignore(); // 清除输入缓冲区

    if (choice == 1) {
        cout << "请输入文件名: ";
        string filename;
        getline(cin, filename);

        if (!processor.loadFromFile(filename)) {
            cout << "使用示例文本进行演示..." << endl;
            // 创建示例文本
            ofstream example("example.txt");
            example << "The quick brown fox jumps over the lazy dog.\n";
            example << "Programming is the art of telling another human what one wants the computer to do.\n";
            example << "STL provides a set of common classes for C++ programming.\n";
            example << "Algorithms are the heart of computer science.\n";
            example << "The standard template library makes programming easier and more efficient.\n";
            example.close();

            processor.loadFromFile("example.txt");
        }
    } else {
        processor.loadFromInput();
    }

    // 处理文本
    processor.processText();
}

```

```
// 显示各种分析结果
processor.showStatistics();
processor.showTopWords(10);

// 交互式查询
cout << "\n请输入要搜索的单词: ";
string searchWord;
getline(cin, searchWord);
processor.findLinesWithWord(searchWord);

processor.findLongWords(6);
processor.generateWordCloud();

return 0;
}
```

8.2 学生成绩管理系统

cpp

```
#include <iostream>
#include <vector>
#include <map>
#include <string>
#include <algorithm>
#include <numeric>
#include <iomanip>
#include <fstream>
using namespace std;

class Student {
public:
    string id;
    string name;
    map<string, double> scores; // 科目 -> 分数

    Student(const string& id, const string& name)
        : id(id), name(name) {}

    void addScore(const string& subject, double score) {
        scores[subject] = score;
    }

    double getAverageScore() const {
        if (scores.empty()) return 0.0;

        double sum = accumulate(scores.begin(), scores.end(), 0.0,
            [](double acc, const pair<string, double>& p) {
                return acc + p.second;
            });
        return sum / scores.size();
    }

    double getTotalScore() const {
        return accumulate(scores.begin(), scores.end(), 0.0,
            [](double acc, const pair<string, double>& p) {
                return acc + p.second;
            });
    }

    char getGrade() const {
        double avg = getAverageScore();
        if (avg >= 90) return 'A';
        if (avg >= 80) return 'B';
        if (avg >= 70) return 'C';
        if (avg >= 60) return 'D';
    }
};
```

```

        return 'F';
    }
};

class GradeManager {
private:
    vector<Student> students;
    set<string> subjects;

public:
    void addStudent(const string& id, const string& name) {
        // 检查学号是否已存在
        auto it = find_if(students.begin(), students.end(),
            [&id](const Student& s) { return s.id == id; });

        if (it != students.end()) {
            cout << "学号 " << id << " 已存在! " << endl;
            return;
        }

        students.emplace_back(id, name);
        cout << "成功添加学生: " << name << " (学号: " << id << ")" << endl;
    }

    void addScore(const string& id, const string& subject, double score) {
        auto it = find_if(students.begin(), students.end(),
            [&id](Student& s) { return s.id == id; });

        if (it == students.end()) {
            cout << "未找到学号为 " << id << " 的学生! " << endl;
            return;
        }

        if (score < 0 || score > 100) {
            cout << "分数必须在0-100之间! " << endl;
            return;
        }

        it->addScore(subject, score);
        subjects.insert(subject);
        cout << "成功添加成绩: " << it->name << " - " << subject << ": " << score << endl;
    }

    void displayAllStudents() const {
        if (students.empty()) {
            cout << "没有学生数据! " << endl;
            return;
        }
    }
};

```



```

}

cout << "\n=== 所有学生信息 ===" << endl;
cout << setw(10) << "学号" << setw(15) << "姓名"
    << setw(10) << "平均分" << setw(8) << "等级" << endl;
cout << string(43, '-') << endl;

for (const Student& student : students) {
    cout << setw(10) << student.id
        << setw(15) << student.name
        << setw(10) << fixed << setprecision(2) << student.getAverageScore()
        << setw(8) << student.getGrade() << endl;
}
}

void displayDetailedReport(const string& id) const {
    auto it = find_if(students.begin(), students.end(),
        [&id](const Student& s) { return s.id == id; });

    if (it == students.end()) {
        cout << "未找到学号为 " << id << " 的学生！ " << endl;
        return;
    }

    cout << "\n=== " << it->name << " 的详细成绩单 ===" << endl;
    cout << "学号: " << it->id << endl;
    cout << "姓名: " << it->name << endl;
    cout << "\n各科成绩:" << endl;

    for (const auto& score : it->scores) {
        cout << " " << setw(12) << score.first << ": "
            << setw(6) << fixed << setprecision(1) << score.second << endl;
    }

    cout << "\n总分: " << fixed << setprecision(1) << it->getTotalScore() << endl;
    cout << "平均分: " << fixed << setprecision(2) << it->getAverageScore() << endl;
    cout << "等级: " << it->getGrade() << endl;
}

void rankStudents() const {
    if (students.empty()) {
        cout << "没有学生数据！ " << endl;
        return;
    }

    vector<Student> sortedStudents = students;
    sort(sortedStudents.begin(), sortedStudents.end(),

```

```
[(const Student& a, const Student& b) {  
    return a.getAverageScore() > b.getAverageScore();  
}];
```

```
cout << "\n=== 学生排名（按平均分） ===" << endl;  
cout << setw(5) << "排名" << setw(10) << "学号" << setw(15) << "姓名"  
    << setw(10) << "平均分" << setw(8) << "等级" << endl;  
cout << string(48, '-') << endl;
```

```
for (size_t i = 0; i < sortedStudents.size(); ++i) {  
    const Student& student = sortedStudents[i];  
    cout << setw(5) << (i + 1)  
        << setw(10) << student.id  
        << setw(15) << student.name  
        << setw(10) << fixed << setprecision(2) << student.getAverageScore()  
        << setw(8) << student.getGrade() << endl;  
}  
}
```

```
void subjectStatistics() const {  
    if (subjects.empty()) {  
        cout << "没有科目数据！" << endl;  
        return;  
    }
```

```
cout << "\n=== 各科目统计 ===" << endl;  
cout << setw(12) << "科目" << setw(8) << "人数" << setw(10) << "平均分"  
    << setw(10) << "最高分" << setw(10) << "最低分" << endl;  
cout << string(50, '-') << endl;
```

```
for (const string& subject : subjects) {  
    vector<double> scores;
```

// 收集该科目的所有分数

```
for (const Student& student : students) {  
    auto it = student.scores.find(subject);  
    if (it != student.scores.end()) {  
        scores.push_back(it->second);  
    }  
}
```

```
if (!scores.empty()) {  
    double avg = accumulate(scores.begin(), scores.end(), 0.0) / scores.size();  
    double max_score = *max_element(scores.begin(), scores.end());  
    double min_score = *min_element(scores.begin(), scores.end());  
  
    cout << setw(12) << subject
```

```

        << setw(8) << scores.size()
        << setw(10) << fixed << setprecision(2) << avg
        << setw(10) << fixed << setprecision(1) << max_score
        << setw(10) << fixed << setprecision(1) << min_score << endl;
    }
}

void findTopStudents(int n = 3) const {
    if (students.empty()) {
        cout << "没有学生数据! " << endl;
        return;
    }

    vector<Student> sortedStudents = students;
    sort(sortedStudents.begin(), sortedStudents.end(),
        [](const Student& a, const Student& b) {
            return a.getAverageScore() > b.getAverageScore();
        });

    int displayCount = min(n, static_cast<int>(sortedStudents.size()));

    cout << "\n=== 前 " << displayCount << " 名学生 ===" << endl;
    for (int i = 0; i < displayCount; ++i) {
        const Student& student = sortedStudents[i];
        cout << "第 " << (i + 1) << " 名: " << student.name
            << " (学号: " << student.id << ")"
            << ", 平均分: " << fixed << setprecision(2) << student.getAverageScore()
            << ", 等级: " << student.getGrade() << endl;
    }
}

void saveToFile(const string& filename) const {
    ofstream file(filename);
    if (!file.is_open()) {
        cout << "无法创建文件 " << filename << endl;
        return;
    }

    file << "学号,姓名";
    for (const string& subject : subjects) {
        file << "," << subject;
    }
    file << ",平均分,等级\n";

    for (const Student& student : students) {
        file << student.id << "," << student.name;
    }
}

```

```

        for (const string& subject : subjects) {
            auto it = student.scores.find(subject);
            if (it != student.scores.end()) {
                file << "," << it->second;
            } else {
                file << ",";
            }
        }

        file << "," << fixed << setprecision(2) << student.getAverageScore()
            << "," << student.getGrade() << "\n";
    }

    file.close();
    cout << "数据已保存到 " << filename << endl;
}

void searchStudent(const string& keyword) const {
    vector<const Student*> found;

    for (const Student& student : students) {
        if (student.id.find(keyword) != string::npos ||
            student.name.find(keyword) != string::npos) {
            found.push_back(&student);
        }
    }

    if (found.empty()) {
        cout << "未找到包含 \" << keyword << "\" 的学生" << endl;
        return;
    }

    cout << "\n=== 搜索结果 ===" << endl;
    for (const Student* student : found) {
        cout << "学号: " << student->id << ", 姓名: " << student->name
            << ", 平均分: " << fixed << setprecision(2) << student->getAverageScore()
            << ", 等级: " << student->getGrade() << endl;
    }
}

};

// 演示程序
int main() {
    GradeManager manager;

    // 添加示例数据

```

```
cout << "=== 学生成绩管理系统 ===" << endl;
cout << "初始化示例数据..." << endl;

manager.addStudent("2023001", "张三");
manager.addStudent("2023002", "李四");
manager.addStudent("2023003", "王五");
manager.addStudent("2023004", "赵六");
manager.addStudent("2023005", "钱七");

// 添加成绩
vector<string> subjects = {"数学", "英语", "物理", "化学", "计算机"};

// 为每个学生添加随机成绩 (模拟)
vector<vector<double>> scores = {
    {92, 88, 85, 90, 95}, // 张三
    {78, 82, 75, 80, 88}, // 李四
    {85, 90, 88, 85, 92}, // 王五
    {95, 93, 90, 95, 98}, // 赵六
    {70, 75, 68, 72, 80}  // 钱七
};

vector<string> studentIds = {"2023001", "2023002", "2023003", "2023004", "2023005"};

for (int i = 0; i < studentIds.size(); ++i) {
    for (int j = 0; j < subjects.size(); ++j) {
        manager.addScore(studentIds[i], subjects[j], scores[i][j]);
    }
}

// 显示各种报告
manager.displayAllStudents();
manager.rankStudents();
manager.subjectStatistics();
manager.findTopStudents(3);

// 显示详细报告
manager.displayDetailedReport("2023004");

// 搜索功能
manager.searchStudent("张");

// 保存到文件
manager.saveToFile("grades.csv");

cout << "\n=== 交互式操作 ===" << endl;
cout << "1. 添加学生" << endl;
cout << "2. 添加成绩" << endl;
```

```
cout << "3. 查看学生详情" << endl;
cout << "4. 搜索学生" << endl;
cout << "5. 退出" << endl;
```

```
int choice;
string input;
```

```
while (true) {
    cout << "\n请选择操作 (1-5): ";
    cin >> choice;
    cin.ignore();

    switch (choice) {
        case 1: {
            cout << "请输入学号: ";
            string id;
            getline(cin, id);
            cout << "请输入姓名: ";
            string name;
            getline(cin, name);
            manager.addStudent(id, name);
            break;
        }
        case 2: {
            cout << "请输入学号: ";
            string id;
            getline(cin, id);
            cout << "请输入科目: ";
            string subject;
            getline(cin, subject);
            cout << "请输入分数: ";
            double score;
            cin >> score;
            manager.addScore(id, subject, score);
            break;
        }
        case 3: {
            cout << "请输入学号: ";
            string id;
            getline(cin, id);
            manager.displayDetailedReport(id);
            break;
        }
        case 4: {
            cout << "请输入搜索关键词: ";
            string keyword;
            getline(cin, keyword);
```

```
        manager.searchStudent(keyword);
        break;
    }
    case 5:
        cout << "谢谢使用！" << endl;
        return 0;
    default:
        cout << "无效选择，请重试。" << endl;
    }
}

return 0;
}
```

8.3 任务调度系统

cpp

```

#include <iostream>
#include <queue>
#include <vector>
#include <string>
#include <chrono>
#include <thread>
#include <algorithm>
#include <map>
using namespace std;
using namespace chrono;

// 任务类
class Task {
public:
    int id;
    string description;
    int priority;    // 优先级 (1-10, 10最高)
    int duration;    // 预计执行时间 (秒)
    system_clock::time_point created_time;
    system_clock::time_point deadline;    // 截止时间
    bool completed;

    Task(int id, const string& desc, int prio, int dur, int deadline_minutes = 60)
        : id(id), description(desc), priority(prio), duration(dur), completed(false) {
        created_time = system_clock::now();
        deadline = created_time + minutes(deadline_minutes);
    }

    // 用于优先级队列的比较 (优先级高的先执行)
    bool operator<(const Task& other) const {
        if (priority != other.priority) {
            return priority < other.priority; // 注意: priority_queue是大根堆, 所以用<
        }
        return deadline > other.deadline; // 截止时间早的先执行
    }

    bool isOverdue() const {
        return system_clock::now() > deadline && !completed;
    }

    int getWaitingTime() const {
        auto now = system_clock::now();
        return duration_cast<seconds>(now - created_time).count();
    }
};

```


// 任务调度器

```
class TaskScheduler {
```

```
private:
```

```
    priority_queue<Task> taskQueue;    // 待执行任务队列
```

```
    vector<Task> completedTasks;    // 已完成任务列表
```

```
    vector<Task> overdueTasks;    // 超时任务列表
```

```
    int nextTaskId;
```

```
    map<int, int> priorityStats;    // 各优先级任务统计
```

```
public:
```

```
    TaskScheduler() : nextTaskId(1) {}
```

// 添加任务

```
void addTask(const string& description, int priority, int duration, int deadline_minutes = 60) {
```

```
    if (priority < 1 || priority > 10) {
```

```
        cout << "优先级必须在1-10之间! " << endl;
```

```
        return;
```

```
    }
```

```
    Task task(nextTaskId++, description, priority, duration, deadline_minutes);
```

```
    taskQueue.push(task);
```

```
    priorityStats[priority]++;
```

```
    cout << "任务已添加: ID=" << task.id << ", 描述=" << description
```

```
        << ", 优先级=" << priority << ", 预计时间=" << duration << "秒" << endl;
```

```
}
```

// 执行下一个任务

```
bool executeNextTask() {
```

```
    if (taskQueue.empty()) {
```

```
        cout << "没有待执行的任务! " << endl;
```

```
        return false;
```

```
    }
```

```
    Task currentTask = taskQueue.top();
```

```
    taskQueue.pop();
```

// 检查是否超时

```
    if (currentTask.isOverdue()) {
```

```
        cout << "任务 " << currentTask.id << " 已超时, 移至超时列表" << endl;
```

```
        overdueTasks.push_back(currentTask);
```

```
        return true;
```

```
    }
```

```
    cout << "正在执行任务 " << currentTask.id << ": " << currentTask.description << endl;
```

```
    cout << "预计执行时间: " << currentTask.duration << " 秒" << endl;
```

```

cout << "等待时间: " << currentTask.getWaitingTime() << " 秒" << endl;

// 模拟任务执行
this_thread::sleep_for(seconds(min(currentTask.duration, 3))); // 最多等待3秒

currentTask.completed = true;
completedTasks.push_back(currentTask);

cout << "任务 " << currentTask.id << " 执行完成! " << endl;
return true;
}

// 批量执行任务
void executeAllTasks() {
    cout << "\n=== 开始批量执行任务 ===" << endl;

    while (!taskQueue.empty()) {
        executeNextTask();
        cout << "剩余任务数: " << taskQueue.size() << endl << endl;

        // 模拟间隔
        this_thread::sleep_for(milliseconds(500));
    }

    cout << "所有任务执行完毕! " << endl;
}

// 显示待执行任务
void showPendingTasks() const {
    if (taskQueue.empty()) {
        cout << "没有待执行的任务" << endl;
        return;
    }

    // 由于priority_queue不支持迭代，我们需要复制一份来显示
    priority_queue<Task> tempQueue = taskQueue;
    vector<Task> tasks;

    while (!tempQueue.empty()) {
        tasks.push_back(tempQueue.top());
        tempQueue.pop();
    }

    cout << "\n=== 待执行任务列表 ===" << endl;
    cout << "ID\t描述\t\t\t优先级\t时长\t状态" << endl;
    cout << string(60, '-') << endl;

```

```

for (const Task& task : tasks) {
    cout << task.id << "\t"
        << task.description.substr(0, 20) << "\t"
        << task.priority << "\t"
        << task.duration << "s\t";

    if (task.isOverdue()) {
        cout << "超时";
    } else {
        auto remaining = duration_cast<minutes>(task.deadline - system_clock::now()).count();
        cout << "剩余" << remaining << "分钟";
    }
    cout << endl;
}
}

```

// 显示已完成任务

```

void showCompletedTasks() const {
    if (completedTasks.empty()) {
        cout << "没有已完成的任务" << endl;
        return;
    }

    cout << "\n=== 已完成任务列表 ===" << endl;
    cout << "ID\t描述\t\t\t优先级\t时长\t等待时间" << endl;
    cout << string(65, '-') << endl;

```

```

for (const Task& task : completedTasks) {
    cout << task.id << "\t"
        << task.description.substr(0, 20) << "\t"
        << task.priority << "\t"
        << task.duration << "s\t"
        << task.getWaitingTime() << "s" << endl;
}
}

```

// 显示统计信息

```

void showStatistics() const {
    cout << "\n=== 任务统计信息 ===" << endl;
    cout << "待执行任务: " << taskQueue.size() << endl;
    cout << "已完成任务: " << completedTasks.size() << endl;
    cout << "超时任务: " << overdueTasks.size() << endl;

    if (!completedTasks.empty()) {
        // 计算平均等待时间
        int totalWaitTime = accumulate(completedTasks.begin(), completedTasks.end(), 0,
            [](int sum, const Task& task) {

```

```

        return sum + task.getWaitingTime();
    });

    double avgWaitTime = static_cast<double>(totalWaitTime) / completedTasks.size();
    cout << "平均等待时间: " << avgWaitTime << " 秒" << endl;

    // 按优先级统计已完成任务
    map<int, int> completedByPriority;
    for (const Task& task : completedTasks) {
        completedByPriority[task.priority]++;
    }

    cout << "\n各优先级完成情况:" << endl;
    for (const auto& pair : completedByPriority) {
        cout << " 优先级 " << pair.first << ": " << pair.second << " 个任务" << endl;
    }
}

if (!overdueTasks.empty()) {
    cout << "\n超时任务:" << endl;
    for (const Task& task : overdueTasks) {
        cout << " ID " << task.id << ": " << task.description << endl;
    }
}

// 查找特定任务
void findTask(const string& keyword) const {
    vector<Task> found;

    // 在待执行任务中查找
    priority_queue<Task> tempQueue = taskQueue;
    while (!tempQueue.empty()) {
        Task task = tempQueue.top();
        tempQueue.pop();

        if (task.description.find(keyword) != string::npos) {
            found.push_back(task);
        }
    }

    // 在已完成任务中查找
    for (const Task& task : completedTasks) {
        if (task.description.find(keyword) != string::npos) {
            found.push_back(task);
        }
    }
}

```

```
// 在超时任务中查找
```

```
for (const Task& task : overdueTasks) {  
    if (task.description.find(keyword) != string::npos) {  
        found.push_back(task);  
    }  
}  
  
if (found.empty()) {  
    cout << "未找到包含 \" << keyword << \" 的任务" << endl;  
    return;  
}
```

```
cout << "\n=== 搜索结果 ===" << endl;  
for (const Task& task : found) {  
    cout << "ID " << task.id << ": " << task.description  
        << " (优先级: " << task.priority;  
    if (task.completed) {  
        cout << ", 已完成";  
    } else if (task.isOverdue()) {  
        cout << ", 已超时";  
    } else {  
        cout << ", 待执行";  
    }  
    cout << endl;  
}
```

```
// 清理超时任务
```

```
void cleanupOverdueTasks() {  
    cout << "清理了 " << overdueTasks.size() << " 个超时任务" << endl;  
    overdueTasks.clear();  
}  
};
```

```
// 演示程序
```

```
int main() {  
    TaskScheduler scheduler;  
  
    cout << "=== 任务调度系统 ===" << endl;
```

```
// 添加示例任务
```

```
cout << "\n添加示例任务..." << endl;  
scheduler.addTask("处理客户邮件", 8, 5, 30);  
scheduler.addTask("备份数据库", 6, 10, 120);  
scheduler.addTask("系统安全扫描", 9, 15, 60);  
scheduler.addTask("生成月度报告", 5, 8, 180);  
scheduler.addTask("紧急bug修复", 10, 3, 15);
```

```
scheduler.addTask("代码审查", 7, 6, 90);
scheduler.addTask("服务器维护", 4, 20, 240);
```

```
// 显示当前状态
```

```
scheduler.showPendingTasks();
scheduler.showStatistics();
```

```
cout << "\n=== 交互式操作 ===" << endl;
cout << "1. 执行下一个任务" << endl;
cout << "2. 批量执行所有任务" << endl;
cout << "3. 添加新任务" << endl;
cout << "4. 查看待执行任务" << endl;
cout << "5. 查看已完成任务" << endl;
cout << "6. 查看统计信息" << endl;
cout << "7. 搜索任务" << endl;
cout << "8. 清理超时任务" << endl;
cout << "9. 退出" << endl;
```

```
int choice;
```

```
while (true) {
```

```
    cout << "\n请选择操作 (1-9): ";
```

```
    cin >> choice;
```

```
    cin.ignore();
```

```
    switch (choice) {
```

```
        case 1:
```

```
            scheduler.executeNextTask();
```

```
            break;
```

```
        case 2:
```

```
            scheduler.executeAllTasks();
```

```
            break;
```

```
        case 3: {
```

```
            cout << "请输入任务描述: ";
```

```
            string desc;
```

```
            getline(cin, desc);
```

```
            cout << "请输入优先级 (1-10): ";
```

```
            int priority;
```

```
            cin >> priority;
```

```
            cout << "请输入预计执行时间 (秒): ";
```

```
            int duration;
```

```
            cin >> duration;
```

```
            cout << "请输入截止时间 (分钟): ";
```

```
            int deadline;
```

```
            cin >> deadline;
```

```
            scheduler.addTask(desc, priority, duration, deadline);
```

```
            break;
```

```
}
```

```
case 4:
    scheduler.showPendingTasks();
    break;
case 5:
    scheduler.showCompletedTasks();
    break;
case 6:
    scheduler.showStatistics();
    break;
case 7: {
    cout << "请输入搜索关键词: ";
    string keyword;
    getline(cin, keyword);
    scheduler.findTask(keyword);
    break;
}
case 8:
    scheduler.cleanupOverdueTasks();
    break;
case 9:
    cout << "谢谢使用任务调度系统! " << endl;
    return 0;
default:
    cout << "无效选择, 请重试。" << endl;
}
}

return 0;
}
```

附录

A. STL算法速查表

A.1 非修改性算法

算法	功能	示例
all_of	检查是否所有元素满足条件	all_of(v.begin(), v.end(), [](int x){ return x > 0; })
any_of	检查是否有元素满足条件	any_of(v.begin(), v.end(), [](int x){ return x < 0; })
none_of	检查是否没有元素满足条件	none_of(v.begin(), v.end(), [](int x){ return x == 0; })
for_each	对每个元素执行操作	for_each(v.begin(), v.end(), [](int x){ cout << x << " "; })
find	查找值	find(v.begin(), v.end(), 42)
find_if	按条件查找	find_if(v.begin(), v.end(), [](int x){ return x > 10; })
count	计数特定值	count(v.begin(), v.end(), 5)
count_if	按条件计数	count_if(v.begin(), v.end(), [](int x){ return x % 2 == 0; })

A.2 修改性算法

算法	功能	示例
copy	复制元素	copy(src.begin(), src.end(), dest.begin())
copy_if	按条件复制	copy_if(src.begin(), src.end(), back_inserter(dest), pred)
transform	变换元素	transform(v.begin(), v.end(), v.begin(), [](int x){ return x*2; })
fill	填充值	fill(v.begin(), v.end(), 42)
replace	替换值	replace(v.begin(), v.end(), old_val, new_val)
remove	移除值	remove(v.begin(), v.end(), value)
unique	去重	unique(v.begin(), v.end())
reverse	反转	reverse(v.begin(), v.end())

A.3 排序算法

算法	功能	示例
sort	排序	sort(v.begin(), v.end())
stable_sort	稳定排序	stable_sort(v.begin(), v.end(), comp)
partial_sort	部分排序	partial_sort(v.begin(), v.begin()+3, v.end())
nth_element	第n个元素	nth_element(v.begin(), v.begin()+n, v.end())
binary_search	二分查找	binary_search(v.begin(), v.end(), value)

B. 容器特性对比表

容器	随机访问	插入/删除(头)	插入/删除(尾)	插入/删除(中间)	内存开销
vector	O(1)	O(n)	O(1)*	O(n)	低
deque	O(1)	O(1)	O(1)	O(n)	中等
list	O(n)	O(1)	O(1)	O(1)	高
set	O(log n)	O(log n)	O(log n)	O(log n)	中等
map	O(log n)	O(log n)	O(log n)	O(log n)	中等

*注：vector在尾部插入可能需要重新分配内存，平摊复杂度为O(1)

C. 迭代器类型速查

迭代器类型	支持操作	典型容器
输入迭代器	*it, ++it, ==, !=	istream_iterator
输出迭代器	*it = val, ++it	ostream_iterator
前向迭代器	输入+输出+多次遍历	forward_list
双向迭代器	前向+--it	list, set, map
随机访问迭代器	双向+it[n], it+n, it-n	vector, deque, array

D. 函数对象速查

D.1 算术函数对象

```
cpp
plus<T>()      // +
minus<T>()     // -
multiplies<T>() // *
divides<T>()   ///
modulus<T>()   // %
negate<T>()    // -x
```

D.2 比较函数对象

```
cpp
equal_to<T>()   // ==
not_equal_to<T>() // !=
greater<T>()    // >
less<T>()       // <
greater_equal<T>() // >=
less_equal<T>()  // <=
```

D.3 逻辑函数对象

```
cpp

logical_and<T>() //&&
logical_or<T>()  //||
logical_not<T>() //!
```

E. 常用STL头文件

头文件	内容
<vector>	vector容器
<list>	list容器
<deque>	deque容器
<set>	set, multiset容器
<map>	map, multimap容器
<stack>	stack适配器
<queue>	queue, priority_queue适配器
<algorithm>	大部分STL算法
<numeric>	数值算法
<iterator>	迭代器工具
<functional>	函数对象
<string>	string类
<memory>	智能指针

F. 最佳实践总结

F.1 容器选择指南

- 1. 需要随机访问 → vector
- 2. 频繁插入/删除中间元素 → list
- 3. 需要在两端操作 → deque
- 4. 需要排序和唯一性 → set
- 5. 需要键值对应关系 → map
- 6. 特定访问模式 → stack / queue / priority_queue

F.2 性能优化建议

- 1. 预分配容量：使用避免频繁重新分配 reserve()
- 2. 选择合适的容器：根据操作特点选择最适合的容器
- 3. 使用移动语义：优先使用和移动构造 std::move

4. 避免不必要的复制：使用引用和const引用
5. 使用范围for循环：代码更简洁，性能更好

F.3 内存管理建议

1. 优先使用智能指针：\、 `unique_ptr` `shared_ptr` `weak_ptr`
2. 避免裸指针：除非与C接口交互
3. 注意循环引用：使用打破循环 `weak_ptr`
4. **RAII**原则：资源在构造时获取，析构时释放

F.4 编码风格建议

```
cpp

// 好的做法
auto it = find_if(container.begin(), container.end(),
    [](const auto& item) { return item.isValid(); });

if (it != container.end()) {
    // 找到了有效元素
    process(*it);
}

// 使用范围for循环
for (const auto& item : container) {
    if (item.needsProcessing()) {
        process(item);
    }
}

// 使用STL算法替代手写循环
transform(input.begin(), input.end(), output.begin(),
    [](int x) { return x * x; });
```

G. 学习路径建议

初学者（1-2周）

1. 学习类的基本使用 `string`
2. 掌握容器的增删改查 `vector`
3. 了解基本的迭代器概念
4. 学习简单的算法：\、 `sort` `find` `count`

进阶（2-3周）

1. 学习其他序列容器：、`list` `deque`
2. 掌握关联容器：、`set` `map`
3. 了解容器适配器：、`stack` `queue` `priority_queue`
4. 学习更多算法：、`transform` `copy_if` `for_each`

高级 (3-4周)

1. 深入理解迭代器类型和失效
2. 学习函数对象和Lambda表达式
3. 掌握智能指针的使用
4. 学习STL的设计理念和泛型编程

专家级 (持续学习)

1. 研究STL源码实现
2. 学习自定义容器和算法
3. 深入理解模板元编程
4. 学习并发和并行STL算法

H. 常见错误和解决方案

H.1 迭代器失效

```
cpp

// 错误的做法
for (auto it = vec.begin(); it != vec.end(); ++it) {
    if (*it == target) {
        vec.erase(it); // 迭代器失效!
    }
}

// 正确的做法
for (auto it = vec.begin(); it != vec.end(); ) {
    if (*it == target) {
        it = vec.erase(it); // erase返回下一个有效迭代器
    } else {
        ++it;
    }
}

// 更好的做法
vec.erase(remove(vec.begin(), vec.end(), target), vec.end());
```

H.2 容器大小问题

```
cpp

// 错误的做法
vector<int> dest(5);
copy(source.begin(), source.end(), dest.begin()); // 如果source>5个元素会越界

// 正确的做法
vector<int> dest;
copy(source.begin(), source.end(), back_inserter(dest));
```

H.3 智能指针循环引用

```
cpp

// 问题代码
class Parent {
    shared_ptr<Child> child;
};

class Child {
    shared_ptr<Parent> parent; // 循环引用!
};

// 解决方案
class Child {
    weak_ptr<Parent> parent; // 使用weak_ptr
};
```

I. 练习题答案示例

练习1：回文检查器

```
cpp
```

```

bool isPalindrome(const string& str) {
    string cleaned;
    for (char c : str) {
        if (isalnum(c)) {
            cleaned += tolower(c);
        }
    }

    return equal(cleaned.begin(), cleaned.begin() + cleaned.size()/2,
                cleaned.rbegin());
}

```

练习2：单词统计器

```

cpp

map<string, int> countWords(const string& text) {
    map<string, int> wordCount;
    istringstream iss(text);
    string word;

    while (iss >> word) {
        // 清理标点符号
        word.erase(remove_if(word.begin(), word.end(),
                             [](char c) { return !isalnum(c); }),
                   word.end());

        // 转换为小写
        transform(word.begin(), word.end(), word.begin(), ::tolower);

        if (!word.empty()) {
            wordCount[word]++;
        }
    }

    return wordCount;
}

```

练习3：数据排序和去重

```

cpp

```

```
template<typename T>
int reduce(T arr[], int n) {
    // 使用vector来利用STL算法
    vector<T> vec(arr, arr + n);

    // 排序
    sort(vec.begin(), vec.end());

    // 去重
    auto newEnd = unique(vec.begin(), vec.end());
    vec.erase(newEnd, vec.end());

    // 复制回原数组
    copy(vec.begin(), vec.end(), arr);

    return vec.size();
}
```

总结

这份C++ STL完整自学手册涵盖了：

1. **string类**：从基础构造到高级作，包含实际应用案例
2. **智能指针**：内存管理的现代化解决方案
3. **容器详解**：各种容器的特点、用法和选择指南
4. **迭代器深入**：理解STL的核心概念
5. **算法详解**：常用算法的使用和最佳实践
6. **函数对象**：包括Lambda表达式和标准函数对象
7. **实战练习**：三个完整的项目案例
8. **附录资料**：速查表、最佳实践和学习路径

通过学习这份手册，您将能够：

- 熟练使用STL的各种组件
- 编写高质量、高效率的C++代码
- 理解现代C++的编程思想
- 具备解决实际编程问题的能力

建议按照章节顺序学习，每完成一章后进行相应的练习，并尝试将所学知识应用到实际项目中。记住，熟练掌握STL需要大量的实践，不要害怕犯错，在错误中学习才能真正掌握这些强大的工具。

1 迭代器基础概念

迭代器是STL的核心概念，它提供了访问容器元素的统一接口。

CPP


```

#include <iostream>
#include <vector>
#include <list>
#include <iterator>
using namespace std;

int main() {
    vector<int> vec = {1, 2, 3, 4, 5};
    list<int> lst = {1, 2, 3, 4, 5};

    cout << "=== 迭代器基本使用 ===" << endl;

    // vector的迭代器
    cout << "vector遍历: ";
    for (auto it = vec.begin(); it != vec.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // list的迭代器
    cout << "list遍历: ";
    for (auto it = lst.begin(); it != lst.end(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // 反向迭代器
    cout << "vector反向遍历: ";
    for (auto it = vec.rbegin(); it != vec.rend(); ++it) {
        cout << *it << " ";
    }
    cout << endl;

    // 常量迭代器
    cout << "list常量遍历: ";
    for (auto it = lst.cbegin(); it != lst.cend(); ++it) {
        cout << *it << " ";
        // *it = 10; // 错误! 不能修改
    }
    cout << endl;

    return 0;
}

```

5.2 迭代器类型

STL定义了5种迭代器类型，每种都有特定的能力：

CPP

```

#include <iostream>
#include <vector>
#include <list>
#include <set>
#include <iterator>
using namespace std;

void demonstrateIteratorTypes() {
    cout << "=== 迭代器类型演示 ===" << endl;

    // 1. 输入迭代器 - 只读，单次遍历
    cout << "1. 输入迭代器示例 (istream_iterator) :" << endl;
    // 这里用vector模拟，实际的输入迭代器如istream_iterator
    vector<int> input_data = {1, 2, 3};
    auto input_it = input_data.begin();
    cout << "读取: " << *input_it << endl;
    ++input_it;
    cout << "读取: " << *input_it << endl;

    // 2. 输出迭代器 - 只写
    cout << "\n2. 输出迭代器示例 (ostream_iterator) :" << endl;
    ostream_iterator<int> out_it(cout, " ");
    *out_it = 10; ++out_it;
    *out_it = 20; ++out_it;
    *out_it = 30; ++out_it;
    cout << endl;

    // 3. 前向迭代器 - 可读写，多次遍历，只能前进
    cout << "\n3. 前向迭代器示例:" << endl;
    // forward_list的迭代器是前向迭代器
    vector<int> forward_data = {1, 2, 3, 4, 5};
    auto forward_it = forward_data.begin();
    cout << "前向遍历: ";
    while (forward_it != forward_data.end()) {
        cout << *forward_it << " ";
        ++forward_it;
    }
    cout << endl;

    // 4. 双向迭代器 - 可前进后退
    cout << "\n4. 双向迭代器示例:" << endl;
    list<int> bidirectional_data = {1, 2, 3, 4, 5};
    auto bi_it = bidirectional_data.end();
    --bi_it; // 指向最后一个元素
    cout << "双向遍历 (从后往前) : ";
    while (bi_it != bidirectional_data.begin()) {

```

```

    cout << *bi_it << " ";
    --bi_it;
}
cout << *bi_it << endl; // 输出第一个元素

// 5. 随机访问迭代器 - 支持随机访问
cout << "\n5. 随机访问迭代器示例:" << endl;
vector<int> random_data = {10, 20, 30, 40, 50};
auto random_it = random_data.begin();

cout << "random_it[0] = " << random_it[0] << endl;
cout << "random_it[2] = " << random_it[2] << endl;

random_it += 3; // 跳转到第4个元素
cout << "跳转后: " << *random_it << endl;

random_it -= 2; // 向前跳转2个位置
cout << "回退后: " << *random_it << endl;
}

int main() {
    demonstrateIteratorTypes();
    return 0;
}

```

5.3 迭代器的算术运算

CPP

```
#include <iostream>
#include <vector>
#include <list>
using namespace std;

int main() {
    vector<int> vec = {0, 10, 20, 30, 40, 50, 60, 70, 80, 90};

    cout << "=== 随机访问迭代器算术运算 ===" << endl;

    auto it = vec.begin();

    // 基本运算
    cout << "it指向: " << *it << endl;

    it += 3; // 向前移动3个位置
    cout << "it += 3后指向: " << *it << endl;

    it -= 1; // 向后移动1个位置
    cout << "it -= 1后指向: " << *it << endl;

    // 迭代器加法
    auto it2 = it + 2;
    cout << "it + 2指向: " << *it2 << endl;

    // 迭代器减法
    auto it3 = it2 - 1;
    cout << "it2 - 1指向: " << *it3 << endl;

    // 迭代器距离
    auto distance = it2 - it;
    cout << "it2与it的距离: " << distance << endl;

    // 比较运算
    cout << "it < it2: " << (it < it2) << endl;
    cout << "it > it2: " << (it > it2) << endl;

    // 数组式访问
    cout << "it[0] = " << it[0] << endl;
    cout << "it[1] = " << it[1] << endl;
    cout << "it[2] = " << it[2] << endl;

    cout << "\n=== 双向迭代器限制 ===" << endl;
    list<int> lst = {0, 10, 20, 30, 40};
    auto list_it = lst.begin();
```

```
cout << "list_it指向: " << *list_it << endl;

++list_it;
cout << "++list_it后指向: " << *list_it << endl;

--list_it;
cout << "--list_it后指向: " << *list_it << endl;

// 以下操作对list迭代器是非法的:
// list_it += 2; // 错误!
// list_it[1]; // 错误!
// list_it + 2; // 错误!

return 0;
}
```

5.